UNIVERSITY OF INFORMATION TECHNOLOGY, VNU-HCM

FACULTY OF SOFTWARE ENGINEERING

GRADUATION THESIS

MULTI-TENANT QUOTA AND RATE LIMITING SERVICE

Student: Nguyen Trung Kien · Student ID: **[to fill]**

Advisor: **[to fill]**

Ho Chi Minh City, December 2026

Acknowledgments

[To write in your own words.] A suggested outline: thank your advisor, **[name]**, for the guidance on the design and the evaluation; the Faculty of Software Engineering for the training behind this work; and your family and friends for their support.

Abstract

Companies that sell access to their APIs sell quotas by plan, and every call must be checked against its customer's plan in real time. The check must be exact, because quotas are billed; fast, because it sits in front of every call; and shared, because many servers answer for the same customer. Existing gateways treat quotas as best-effort targets, or give up exactness for speed.

This thesis builds a quota and rate limiting service in C++20, gRPC and Redis Cluster, and splits the problem in two.

- **Daily quotas** are decided in Redis by atomic Lua scripts. They are exact, deduplicated by request id, reset once a day by Redis's clock, and correct under plan changes and three overage policies.
- **Burst limits** are served from leases: tokens an instance takes from Redis in chunks and hands out with one atomic operation. Over any window they admit at most N × C beyond an exact limiter, for N instances and a chunk cap C; the bound is proved and measured.
- **Around both:** a deny cache answers customers out of quota without Redis, a plan cache is kept fresh by Pub/Sub, and each plan has a policy for when Redis fails.
- **For many API owners (merchants):** each merchant's customers are named apart from an API key resolved in memory. A management API and a web portal change limits at run time. The quota script keeps each month's billable totals in the same atomic step as its decision, and invoices are made from them. A merchant's backend capacity is a leased bucket with a reserve for paying customers.

The evaluation tests the two halves of the claim separately, on one laptop.

- **Exactness:** quotas were exact in 20 of 20 adversarial runs on four instances, and 22.6 million checks matched a reference model.
- **The bound:** over-admission measured 34.7, 189.0 and 381.4 tokens against bounds of 40, 200 and 400, within 0.3 tokens of a model.
- **Speed:** a leased check costs about as much as gRPC alone: up to 105,000 checks per second per instance, 400,000 with batches of 10, and a p99 of 0.58 ms at 10,000 per second. A full quota check makes two Redis round trips: 26,000 to 42,000 per second per instance.
- **Failures:** a slow, silent or killed Redis master affected only its own customers, and no call failed.
- **Merchants:**

- Merchants were isolated, and floods refused by their own limits or by the operator's check rate cost the others almost nothing.
- Limit changes applied on two instances within 16 ms.
- Invoices equalled the metered usage for every customer, also with a master killed.
- Tenancy cost less than the measurement noise, and 10,000 merchants of 100 customers needed 193 bytes of Redis memory per customer.

From these measurements, 1,000,000 checks per second is estimated to need 15 to 50 instances and 17 Redis masters; it was not measured.

Keywords: rate limiting, API quotas, multi-tenancy, Redis Cluster, leases, gRPC, C++.

Tóm tắt

Các công ty bán quyền truy cập API bán hạn mức (quota) theo gói, và mỗi lời gọi phải được kiểm tra với gói của khách hàng ngay lập tức. Việc kiểm tra phải chính xác vì hạn mức được tính tiền, phải nhanh vì nó đứng trước mọi lời gọi, và phải dùng chung vì nhiều máy chủ cùng trả lời cho một khách hàng. Các cổng API hiện có coi hạn mức là mục tiêu gần đúng, hoặc đánh đổi độ chính xác lấy tốc độ.

Khóa luận xây dựng một dịch vụ giới hạn hạn mức và tốc độ bằng C++20, gRPC và Redis Cluster, chia bài toán làm hai phần.

- **Hạn mức theo ngày** được quyết định trong Redis bằng các script Lua nguyên tử. Hạn mức chính xác, không đếm trùng nhờ mã yêu cầu, được đặt lại mỗi ngày một lần theo đồng hồ của Redis, và đúng khi khách hàng đổi gói cũng như với ba chính sách vượt hạn mức.
- **Giới hạn tức thời (burst)** được phục vụ từ các "lease": mỗi máy chủ lấy trước từ Redis một phần token và cấp phát cục bộ bằng một thao tác nguyên tử. Trong mọi khoảng thời gian, số lời gọi được cho qua vượt quá một bộ giới hạn chính xác nhiều nhất là N × C, với N máy chủ và kích thước phần C; cận này được chứng minh và đo đạc.
- **Xung quanh hai phần trên:** bộ nhớ đệm từ chối trả lời khách hàng đã hết hạn mức mà không cần hỏi Redis, bộ nhớ đệm gói được cập nhật qua Pub/Sub, và mỗi gói có chính sách riêng khi Redis gặp sự cố.
- **Cho nhiều chủ API (merchant):** khách hàng của mỗi merchant được đặt tên riêng, nhờ khóa API được tra cứu ngay trong bộ nhớ. Một API quản trị và một cổng web thay đổi giới hạn khi hệ thống đang chạy. Script hạn mức lưu tổng số cần tính tiền của mỗi tháng trong cùng bước nguyên tử với quyết định, và hóa đơn được lập từ đó. Năng lực xử lý của backend mỗi merchant là một bucket qua lease, có phần dự trữ cho khách hàng trả phí.

Phần đánh giá kiểm tra riêng từng nửa của luận điểm, trên một máy tính xách tay.

- **Độ chính xác:** hạn mức chính xác trong 20/20 lần chạy với bốn máy chủ, và 22,6 triệu lượt kiểm tra khớp với một mô hình tham chiếu.
- **Cận vượt mức:** đo được 34,7, 189,0 và 381,4 token so với cận 40, 200 và 400, lệch mô hình dưới 0,3 token.
- **Tốc độ:** một lượt kiểm tra qua lease tốn gần như bằng gRPC không có bộ giới hạn: đến 105.000 lượt/giây mỗi máy chủ, 400.000 khi gộp 10 lượt mỗi lời gọi, p99 0,58 ms ở 10.000 lượt/giây. Một lượt kiểm tra hạn mức đầy đủ cần hai lần hỏi Redis: 26.000 đến 42.000 lượt/giây mỗi máy chủ.
- **Sự cố:** một nút Redis chậm, im lặng hoặc bị tắt chỉ ảnh hưởng đến khách hàng của chính nút đó, và không lời gọi nào thất bại.
- **Merchant:**
  - Các merchant được cô lập với nhau, và những đợt gọi dồn dập bị chặn bởi giới hạn của chính merchant hay bởi giới hạn số lượt kiểm tra do nhà vận hành đặt gần như không ảnh hưởng đến merchant khác.
  - Thay đổi giới hạn có hiệu lực trên hai máy chủ trong vòng 16 ms.
  - Hóa đơn khớp với lượng sử dụng đo được của mọi khách hàng, kể cả khi một nút Redis chính bị tắt.
  - Chi phí của cơ chế đa merchant nhỏ hơn độ nhiễu của phép đo, và 10.000 merchant mỗi merchant 100 khách hàng cần 193 byte bộ nhớ Redis cho mỗi khách hàng.

Từ các số đo này, mức 1.000.000 lượt/giây được ước tính cần 15 đến 50 máy chủ và 17 nút Redis chính; con số này là ước tính, chưa được đo.

Từ khóa: giới hạn tốc độ, hạn mức API, đa khách hàng, Redis Cluster, lease, gRPC, C++.

Abbreviations

Abbreviation	Meaning
API	Application Programming Interface
ASan, TSan	AddressSanitizer, ThreadSanitizer
CAS	compare-and-swap, an atomic CPU instruction
gRPC	an RPC framework over HTTP/2, from Google
HTTP	Hypertext Transfer Protocol
LTO, PGO	link-time optimisation, profile-guided optimisation
N × C	instances times chunk cap: the bound on a lease's over-admission
p50, p99, p999	the 50th, 99th and 99.9th percentiles of latency
RPC	remote procedure call
TTL	time to live (of a cache entry or a Redis key)
USL	Universal Scalability Law
UTC	Coordinated Universal Time

Contents

Acknowledgments 2
Abstract 2
Tóm tắt 3
Abbreviations 4
Chapter 1. Introduction 7
1.1 Context: selling access to APIs 7
1.2 Problem 7
1.3 Goals 8
1.4 Approach 9
1.5 Contributions 10
1.6 Structure of the thesis 11
Chapter 2. Background and related work 12
2.1 Rate limiting algorithms 12
2.2 Redis 13
2.3 gRPC and Protocol Buffers 15
2.4 Related systems 16
2.5 Summary 19
Chapter 3. Design 20
3.1 Requirements 20
3.2 Architecture 24
3.3 Data model 27
3.4 Quota and overage logic 30
3.5 Lease protocol and the accuracy bound 33
3.6 Deny cache and idempotency 36
3.7 Failure policies and failure modes 38
3.8 Capacity math 41
3.9 Merchants: one service, many API owners 44
Chapter 4. Implementation 51
4.1 Project structure and build 51
4.2 Lua scripts 52
4.3 Redis client and batching 53
4.4 The lease table 54
4.5 The gRPC server 55

4.6 Reliability 56
4.7 Merchants and the control plane 57
4.8 Testing 59
Chapter 5. Evaluation 61
5.1 Setup 61
5.2 Quota accuracy (evaluation test 1) 61
5.3 Plan changes, the reset and overage (evaluation tests 2 and 3) 62
5.4 The burst accuracy bound and its model (evaluation tests 4 and 12) 63
5.5 Floods and many customers (evaluation tests 5 and 6) 64
5.6 Strict against leased, and scaling (evaluation tests 7 and 8) 64
5.7 Faults (evaluation test 9) 67
5.8 A noisy neighbour (evaluation test 11) 67
5.9 Ablation (evaluation test 10) 68
5.10 Profiling and speed-ups 69
5.11 Extrapolation to 1,000,000 checks per second 70
5.12 Merchants (G10 to G14) 72
5.13 Discussion and threats to validity 76
Chapter 6. Conclusion 78
6.1 Results against the goals 78
6.2 Limitations 80
6.3 Future work 81
References 82

Chapter 1. Introduction

1.1 Context: selling access to APIs

A growing share of software is sold not as an installed product but as an API that customers call over the network. Providers of large language models, SMS and one-time-password gateways, maps, payments and market data all follow the same commercial pattern: a customer signs up, receives an API key, chooses a plan, and pays for a quantity of calls. A free plan might allow 200 calls per day, a paid plan 2,000, an enterprise contract a million. On top of the daily quantity, most plans also limit how fast calls may arrive, for example 2 calls per second on the free plan, both to protect the provider's backend and to stop one customer's burst from degrading service for the others.

Every call therefore raises a question that must be answered before the call is served, in real time and on the request path: *has this customer already used what they paid for?* The component that answers it is a rate limiter, or more precisely a quota service: it holds each customer's usage, compares it with the customer's plan, and returns allow or deny. Because it sits in front of every API call, its answer has two properties that pull in opposite directions. It must be correct, because the quota is the thing being sold. And it must be cheap, because it is paid on every call, including the vast majority of calls from customers who are nowhere near their limit.

Most API owners do not want to build that component, so it can itself be sold as a service. Three parties then meet in it:

- **The platform** runs the service and its Redis cluster, and admits API owners.
- **A merchant** is an API owner that buys the service. It defines its own plans (Free, Pay, Enterprise, with its own numbers and prices), signs up its own customers, and calls the service from its API gateway before serving each call.
- **A customer** is one of the merchant's users, identified by an id the merchant chooses. It never calls the service directly; its calls reach it through the merchant's gateway.

1.2 Problem

Correctness. A quota is a promise in both directions. If the limiter admits more calls than the plan allows, the provider gives away the product it sells; if it admits fewer, it blocks a paying customer, who sees errors for service they paid for. Exactness is harder than it looks once the service runs as several instances behind a load balancer. Two instances that each read "199 used" and each admit one more call have admitted 201. A client that retries a call after a timeout must not be charged twice. A customer who upgrades at noon expects the higher limit on the next call, not after a cache expires. And every instance must agree on when "today" ends, or a customer can be reset twice or not at all.

Widely used managed gateways do not promise exactness. Amazon API Gateway documents that "usage plan throttling and quotas are not hard limits, and are applied on a best-effort basis. In some cases, clients can exceed the quotas that you set" [1], and advises against relying on them to control costs. Kong's rate limiting plugin offers a `local` policy

that keeps counters in each node's memory and, unless a consistent-hashing load balancer sits in front, "diverges when scaling the number of nodes", and a `redis` policy that is accurate only when its counters are synchronised on every request (`sync_rate` set to 0) [3]. Approximation is a reasonable choice for protecting a backend from overload; it is not a reasonable way to enforce a plan a customer pays for.

Throughput. The obvious way to be exact is to keep one counter per customer in a shared store and update it atomically on every call. Every check then costs a network round trip to the store, and the store sees every API call of every customer. At the scale of a large API platform, on the order of a million checks per second, that round trip dominates both the latency added to each API call and the cost of running the limiter. The store becomes the bottleneck, and the customers who make most of the traffic, those far below their limits, pay the full price of exactness they do not need on most of their calls.

Many merchants. Selling the limiter to many API owners adds four demands. One merchant must not see, change or slow down another's customers, though they share the cluster. A merchant changes limits while its customers are calling: an upgrade, a boost in an emergency, a suspension. It bills its customers for what they used, so the numbers on an invoice must be the numbers the limiter decided with, also when a Redis master fails. And a merchant's backend has a capacity of its own: quotas sold per day can arrive in the same second, and when they do, the merchant would rather turn away its free users than its paying ones.

The problem this thesis addresses is how to be **exact where it matters, the daily quota, while being fast where it matters, the high-volume burst check**, and how to make the price of the speed, a bounded inaccuracy, explicit and measured rather than unknown. Then, how to offer that limiter to many merchants as a service, isolated from each other, with invoices that equal the metered usage.

1.3 Goals

The goals come from the two halves of the problem (`docs/DESIGN.md`, section 1); G10 to G14 come from selling the service to many merchants (`docs/PRODUCT_PLAN.md`, section 7).

Table 1.1: The goals of this thesis.

Goal	Statement
G1	No over-admission of a daily quota, under concurrent calls to any number of limiter instances.
G2	A retried call (same request id within five minutes) is counted once.
G3	A plan change applies on the customer's next call, keeping the day's usage.
G4	One reset moment for every customer and instance: 00:00 UTC by the Redis clock.
G5	About 1,000,000 checks per second across a cluster, reached by adding instances and Redis masters.
G6	Server-side latency per check: p50 below 500 µs, p99 below 2 ms.
G7	Over-admission of the burst limit bounded by N × C per refill interval (N instances,

Goal	Statement
	lease chunk C), and measured.
G8	Over-quota floods refused locally; an overloaded instance sheds load explicitly.
G9	Documented behaviour when Redis is slow or unavailable, per plan.
G10	One merchant cannot read, change or slow another's limits.
G11	A limit change or boost applies on every instance within 1 s.
G12	An invoice equals the metered usage exactly, retries included.
G13	Billed overage never exceeds the spending cap.
G14	A merchant's backend never receives more than its capacity plus N × C, and paying customers keep their reserve under a flood of Free calls.

G5 is a cluster-wide figure that a single laptop cannot produce. It is reached in this thesis by measuring the throughput of one instance and of one Redis master, and extrapolating with every assumption stated (Chapter 5); it is never reported as a measurement.

In scope for merchants are the management API, the merchant portal, invoices made from the metered usage, and charging them through a payment provider's sandbox. Out of scope are a payment business (taxes, refunds, collecting debts), multi-region deployment, and a customer-facing sign-up: the merchant signs up its own customers.

1.4 Approach

The thesis builds a quota and rate limiting service in C++20, gRPC and Redis Cluster that splits the problem along the line between the two halves.

The **daily quota** is checked strictly: every call runs one Lua script in Redis that reads the customer's usage for the current UTC day, compares it with the limit from the customer's current plan, and increments it, atomically. Redis runs a script to completion before any other command [11], so two limiter instances can never both take the last unit of a quota. All of one customer's keys share a Redis Cluster hash tag and therefore live on one master [12], which keeps each check to a single round trip to a single node while the cluster as a whole scales out.

The **burst limit** is served from local leases. Each limiter instance takes a chunk of a customer's token bucket from Redis and hands it out with lock-free atomic operations in its own memory; only when the chunk runs low does it ask Redis for another, in a pipelined refill. A burst check that finds tokens costs one compare-and-swap and no network round trip. The price is that tokens held by one instance are invisible to the others: with N instances and chunk size C, at most N × C tokens can be outside Redis, which bounds the over-admission per refill interval. The idea of handing out capacity as leases goes back to Gray and Cheriton's leases for cache consistency [10] and appears in production rate limiting in YouTube's Doorman [9]; this thesis applies it to a server-side token bucket and measures the resulting bound.

A server-side composite check combines the two for one API call: suspended accounts and customers already known to be over quota are refused locally, the burst limit is checked

next, and only calls that pass the burst check reach the strict quota script. In the version built, a customer's burst limit is still one script per check; leasing it per customer is future work (Section 3.2).

Merchants are a layer around this core, not a change to it. A call carries its merchant's API key; the service resolves it to the merchant from an in-memory index and names the customer `<merchant>|<customer>` before the engines see it, so every key, lease and cached denial below is the merchant's own. A control plane writes merchants' plans, limits and boosts to Redis and announces each change, as the plan change above does. The quota script keeps each month's billable totals in the same atomic step as the decision, and invoices are made from them. A merchant's capacity is one more token bucket served from leases, split into a share for every plan and a reserve for plans with a fee.

1.5 Contributions

1. **A quota design that is exact under concurrency, retries and plan changes.** Daily counters keyed by the UTC day taken from the Redis clock, storing `used` rather than `remaining` so that plan changes rewrite nothing, with request-id deduplication and three overage policies (hard, grace, capped billable overage). A Free customer is stopped at exactly 200 of 400 concurrent calls through gRPC on a three-master cluster, and quotas were exact in 20 of 20 adversarial runs on four instances (5.2). A retried call is counted once, and request 2,001 of a plan of 2,000 a day gets three different answers under the three overage policies, tested through gRPC (5.3).
2. **A lease-based fast path with a derived and measured accuracy bound.** Cache-line-aligned leases in a lock-free table with single-flight refills, chunk sizes capped at a fraction of the limit, and pipelined Redis refills. The bound N × C is derived in Chapter 3 and measured there: on four instances at chunk caps of 1%, 5% and 10% the worst case stays below it, as a model of the protocol predicts to within 0.3 tokens (5.4).
3. **An evaluation that tests each half of the claim separately.** Exactness tests (concurrency, day rollover, plan changes, overage), then performance and robustness tests (strict against leased mode, scaling across instances, floods, key cardinality, Redis failures), with reproducible scripts and raw results. Every table of Chapter 5 is generated by a script from the raw results, and the estimate for 1,000,000 checks per second is derived from measured per-instance figures with every assumption written out (5.11).
4. **A multi-tenant service around the limiter, with metering in the decision.** Merchants isolated by name rather than by deployment; limits, boosts and capacities changed at run time; invoices made from totals the quota script keeps atomically with each decision; and a merchant's capacity with a reserve for paying customers, served from leases. Each is evaluated on the real limiter: isolation and a noisy merchant, the latency of a change, invoices against the counters with a master killed, the capacity under a Free flood, 10,000 merchants, and the cost of tenancy (5.12).

1.6 Structure of the thesis

Chapter 2 introduces the background: rate limiting algorithms, the Redis properties the design relies on (single-threaded atomic scripts, cluster hash slots, failover), gRPC and Protocol Buffers, and existing rate limiting systems compared in one table. Chapter 3 presents the design: requirements, architecture, data model, quota and overage logic, the lease protocol with the derivation of its accuracy bound, failure policies, and the capacity estimate for one million checks per second, then merchants (3.9). Chapter 4 describes the implementation: the Lua scripts, the Redis client and its measured selection, the lease table, the gRPC server, merchants and the control plane, and the test approach. Chapter 5 evaluates the system against each goal with measured results and their limitations, merchants in 5.12. Chapter 6 concludes with the results against the goals, the limitations, and future work.

Chapter 2. Background and related work

This chapter introduces what the design in Chapter 3 builds on: the algorithms used to limit request rates (2.1), the properties of Redis that make an exact distributed counter possible (2.2), the gRPC and Protocol Buffers stack that carries every check (2.3), and the existing systems that solve related problems (2.4). Section 2.5 states what those systems leave open.

2.1 Rate limiting algorithms

A rate limiter decides, for each arriving request of a client, whether the request fits within a limit of the form “at most L units per period T”. The algorithms differ in what they remember and therefore in how they behave at the edges of a period.

2.1.1 Fixed window counter

Time is cut into consecutive windows of length T aligned to a fixed origin (for example the Unix epoch), and each client has one counter per window. A request is admitted if the counter is below L, and the counter is incremented. State is one integer per client, and the counter can simply expire with its window.

The cost of this simplicity is the window edge. A client that sends L requests at the end of one window and L more at the start of the next has sent 2L requests in an interval of a few milliseconds, all of them admitted. The project’s script tests reproduce this directly: `WindowEdgeBurstFixedAdmitsTwiceTheLimitSlidingDoesNot` sends L = 10 requests 150 ms before a one-second window boundary and 10 more 20 ms after it: the fixed window admits all 20 within about 170 ms, the sliding window (2.1.2) at most one of the second ten.

2.1.2 Sliding window

A sliding window log stores the timestamp of every admitted request and admits a new one if fewer than L timestamps fall within the last T. It is exact but needs memory proportional to L per client, which is prohibitive for large limits.

The sliding window counter approximates the log with two fixed-window counters, the current and the previous one. The number of requests in the last T is estimated as the current count plus the previous count weighted by the fraction of the previous window that still overlaps the sliding interval. Cloudflare describes this approximation for its rate limiting, noting that it assumes a constant rate during the previous window and “in practice it proved to be good enough” [8]. State is two integers per client, and the 2L edge burst of the fixed window disappears, at the price of a small error when traffic within the previous window was uneven.

2.1.3 Token bucket

A token bucket has a capacity b and a refill rate r. It starts full; each request of cost c takes c tokens if at least c are present and is denied otherwise; tokens are added continuously at rate r up to the capacity. Over any interval of length t the bucket admits at most b + r·t

units, so it allows a burst of size b after a quiet period and a sustained rate r otherwise. The same structure is used in network traffic conditioning, where RFC 2697 defines a marker based on token buckets with a committed and an excess burst size [19], and Amazon API Gateway describes its request throttling in terms of a token bucket with a rate and a burst [2].

A token bucket needs only two numbers of state per client, the token count and the time of the last refill: the tokens due since then can be computed when the next request arrives instead of by a timer. Two implementation details matter for a distributed limiter and recur in Chapter 3. First, "now" must come from one clock for every instance; otherwise two instances with skewed clocks compute different refills for the same bucket. Second, repeated fractional refills accumulate floating-point error unless the state is kept in integers, for example in thousandths of a token.

The leaky bucket, in which requests drain from a queue at a fixed rate, enforces a smooth output rate rather than a burst allowance. It suits traffic shaping more than API quotas, where a customer's burst within their allowance should be served immediately rather than delayed.

2.1.4 The daily quota as a calendar window

A daily quota is a fixed window whose length is one day and whose origin is midnight in an agreed time zone. It is the unit customers buy: "200 calls per day" is easy to state in a price list, easy to show in a response header, and easy to reconcile with a bill. Its edge behaviour, the 2L burst of 2.1.1, is acceptable for a quota because what the customer pays for is the count per day, not the shape of traffic within it; the shape is the job of the second limit.

2.1.5 Why a quota plus a token bucket fits the business model

The two limits answer different questions and tolerate different errors. The **quota** answers "has this customer used what they paid for today?". It is small (hundreds or thousands per day for most customers), it is money, and it must be exact. The **burst limit** answers "is this customer calling too fast right now?". It protects the provider's backend and other customers, it is checked on every call at the full request rate, and an error of a few tokens is harmless as long as it is bounded. A token bucket is the natural burst limit because it needs constant state, allows the short bursts that real clients produce, and is cheap to evaluate. Using the two together lets the design spend exactness only where it is needed, which is the central decision of Chapter 3.

2.2 Redis

Redis is an in-memory data store that keeps all data in RAM and serves it over a simple request/response protocol. Four of its properties carry the design: command execution on a single thread, atomic Lua scripts, a cluster mode that partitions keys by hash slot, and asynchronous replication.

2.2.1 Single-threaded execution and atomic scripts

Redis executes commands one at a time on a single thread (later versions can use extra threads for socket I/O, but not for executing commands). Every command is therefore atomic with respect to every other. A check that has to read a counter, compare it with a limit and write it back is not a single command, though, and doing it as separate commands from a client would let two clients read the same value and both write an increment.

Lua scripts close that gap. A script runs inside the server, and "Redis guarantees the script's atomic execution. While executing the script, all server activities are blocked during its entire runtime" [11]. A read-compare-write of a token bucket or a quota, written as one script, therefore behaves as one indivisible operation however many limiter instances call it. The same blocking is a constraint: a slow script blocks every client of that Redis node, so scripts must do constant work, with no loops over data and no scans.

Two rules of the scripting model shape the client code. First, the server keeps compiled scripts in a cache indexed by the SHA1 digest of their source, and a client invokes a cached script with `EVALSHA` and the digest instead of sending the source with `EVAL` each time. The cache "is always volatile": it is not persisted and may be cleared on restart, on failover to a replica, or by `SCRIPT FLUSH`, in which case `EVALSHA` returns a `NOSCRIPT` error [11]. A client must therefore be prepared to fall back to `EVAL` with the source, which runs the script and caches it again. Second, "all names of keys that a script accesses must be explicitly provided as input key arguments" [11]; a script that builds a key name internally works on a single server but breaks under cluster mode, because the cluster routes the call by its declared keys.

Time inside a script. A limiter needs "now" inside the script, both for token refills and for the current day of a quota. Redis offers the `TIME` command, which returns the server's clock. When scripts were replicated by re-running them on replicas, a non-deterministic command such as `TIME` before a write was forbidden, because a replica would compute a different result. Since Redis 5.0 scripts are replicated by their effects, the write commands they produced, and since Redis 7.0 that is the only mode, so a script may call `TIME` freely [11]. Taking the time from the Redis node that owns the key gives every limiter instance the same clock for that key.

Pipelining. A client may send several commands without waiting for each reply and read the replies afterwards in order. One network round trip then carries many commands, so throughput is no longer bounded by one round trip per call. The project's client spike (`docs/DESIGN.md`, section 3) measured about 290,000 script calls per second from one sending thread over one connection per master, against a three-master cluster on one laptop, by keeping calls in flight instead of waiting for each.

2.2.2 Redis Cluster

Redis Cluster partitions the key space across several master nodes. The key space is split into 16,384 hash slots, and a key's slot is `CRC16(key) mod 16384`; each master owns a subset of the slots [12]. A client that sends a command to the wrong master receives a

`MOVED` reply naming the owner, and updates its slot map; during a live migration of a slot it may receive `ASK`, a one-time redirection [12].

Commands that touch several keys, including a script with several keys, work only if all the keys are in the same slot. **Hash tags** make that controllable: if a key contains a substring in braces, only that substring is hashed [12]. Keys such as `{cust:42}:plan`, `{cust:42}:daily_quota:q` and `{cust:42}:burst:tb` all hash `cust:42`, so all of one customer's state lives on one master and one script may read the plan and update the quota atomically. Distributing customers over masters remains automatic, because different customers have different tags. The cost is that one very hot customer cannot be spread over several masters; Chapter 3 addresses that case with leases.

2.2.3 Replication and failover

Each master can have replicas that receive its writes asynchronously. If a master fails, the cluster promotes one of its replicas after the node timeout. The specification is explicit about the consequence: asynchronous replication leaves "small windows where acknowledged writes can be lost" [12], because a master may acknowledge a write and fail before forwarding it. For a quota counter, this means a failover can lose a few increments, letting a customer make a few more calls than counted; Chapter 3 treats this as a limitation and points to the usage event log as the durable record for billing.

The project's spike also observed what happens without replicas. When one of three masters was stopped, calls for its slots failed at once; after the node timeout, the cluster state became `fail` and every master, including the two healthy ones, answered `CLUSTERDOWN` (with the default `cluster-require-full-coverage yes`); after a restart the node came back without its script cache, so its keys returned `NOSCRIPT` until the client fell back to `EVAL` (`docs/DESIGN.md`, section 3). These observations define the failure cases Chapter 3 must handle.

2.3 gRPC and Protocol Buffers

The limiter is called by API gateways over gRPC, a remote procedure call framework that carries Protocol Buffers messages over HTTP/2.

HTTP/2 multiplexes many concurrent requests as independent streams over one TCP connection [16], so a gateway can keep thousands of checks in flight to a limiter instance without opening thousands of connections, and a slow check does not block the ones behind it. A lost TCP packet still delays every stream on the connection, because HTTP/2 does not address head-of-line blocking in TCP itself [16].

Protocol Buffers define the request and response messages in a schema compiled into C++ and into any client language [15]. Two properties of the format shape the API in Chapter 3. Each field is identified on the wire by its number, not its name, so field numbers must never be reused or renumbered once clients exist; removed fields are declared `reserved`. And in proto3 a field that was never set reads as its zero value, so an enumeration whose zero value meant "allowed" would turn a server bug that forgets to set the reason into an apparent success; the zero value is therefore reserved for "unspecified".

Status codes, deadlines and retries. A gRPC call ends with a status code such as `OK`, `INVALID_ARGUMENT`, `RESOURCE_EXHAUSTED` or `UNAVAILABLE` [14]. A client sets a deadline per call, after which the call fails with `DEADLINE_EXCEEDED` instead of waiting indefinitely [14]. Clients can be configured with a retry policy that retries a call, with exponential backoff, when it fails with one of a list of retryable status codes [13]; `UNAVAILABLE` is the code such policies usually list. This matters for the API design: if the limiter reported "Redis is down" as `UNAVAILABLE`, retry policies would repeat the call against a store that is already struggling. The API in Chapter 3 therefore reports a decision, including "denied because Redis is unreachable", inside an `OK` response, and reserves non-`OK` statuses for calls the limiter could not decide at all.

The C++ callback API. gRPC's C++ server offers an asynchronous callback API in which a handler returns immediately with a reactor object and completes the call later, from any thread, by calling `Finish` exactly once. A check that has to wait for Redis does not occupy a server thread while it waits; it finishes from the Redis client's I/O thread when the reply arrives, and a check served from a local lease finishes inline.

2.4 Related systems

Table 2.1 compares existing systems that enforce request limits. The comparison uses four properties: whether a configured limit is enforced exactly across instances, what a check costs on the hot path, how the system is deployed, and whether its source is open.

Table 2.1: Rate limiting systems compared (from each system's documentation, accessed October 2026).

System	Exact across instances?	Hot-path cost of a check	Deployment	Source
Amazon API Gateway usage plans [1, 2]	No: quotas and throttles are "applied on a best-effort basis"	inside the managed gateway	managed AWS service	proprietary
Kong rate limiting plugin [3, 4]	`local`: no; `cluster`: yes (datastore read and write per request); `redis`: only with `sync_rate` 0	local memory, datastore, or Redis per request	plugin in the Kong gateway	open source (Redis Cluster support in Enterprise)
Envoy ratelimit service [5]	Per Redis counter; fixed windows of a second, minute, hour or day	one gRPC call to the service, which updates Redis; a local cache of over-limit keys avoids Redis for denials	separate Go service called by Envoy	open source
Unkey [6]	Converges: regional decisions, global	regional check, ~30 ms decisions stated	managed service, self-	source available,

System	Exact across instances?	Hot-path cost of a check	Deployment	Source
	convergence within seconds (vendor statement)	by the vendor	hostable	AGPL-3.0
Upstash Ratelimit [7]	Per Redis counter; fixed window recommended for multi-region	client library call to Redis per check	library over (serverless) Redis	open-source SDK
Doorman [9]	Cooperative: clients receive capacity leases and are trusted to respect them	local, within the leased capacity	separate Go service, client libraries	open source, archived 2024
Gubernator [20]	Exact at each limit's owner peer; `GLOBAL` mode trades consistency for scale	forward to the owner peer, batched; with `GLOBAL`, local with asynchronous sync	Go service of peers, state in memory, no Redis	open source (Apache-2.0)
Bucket4j [21]	Exact by default; the `delaying` optimization may over-consume up to a per-node token threshold	remote store per check, or local until the threshold	Java library over Redis, JCache or a database	open source (Apache-2.0)
This thesis	Quota: yes (one atomic script per check). Burst: bounded by N × C, measured	quota: one Redis round trip; burst: one compare-and-swap, Redis only for refills	gRPC service with Redis Cluster	open source (this repository)

Amazon API Gateway attaches usage plans with a throttle (rate and burst) and a quota (requests per day, week or month) to API keys. The documentation states that both "are not hard limits, and are applied on a best-effort basis", that "clients can exceed the quotas that you set", and recommends AWS Budgets and AWS WAF instead of usage plans for controlling costs or access [1]. It is the clearest statement of the gap this thesis addresses: a widely used managed product chooses not to promise exact quotas.

Kong offers three counter policies with an explicit accuracy trade-off [3]. `local` keeps counters in each node's memory, which is fast but diverges as nodes are added; `cluster` stores counters in Kong's datastore with a read and a write per request, which is accurate but has the largest performance cost; `redis` shares counters through Redis and is accurate only when synchronised on every request (`sync_rate` 0). Kong's guidance rules out `local` where every transaction matters, such as financial transactions [3]. The trade-off is the

same one this thesis faces, but Kong chooses one policy per plugin, while this thesis uses the exact path and the fast path for different limits of the same call.

Envoy's ratelimit service is a separate gRPC service that Envoy proxies call with a domain and a set of descriptors; it keeps counters in Redis with limits per second, minute, hour or day [5]. Two of its features have counterparts in this design: an optional local cache of keys already over their limit, which avoids reading Redis again for a client that is being denied (the deny cache in Chapter 3), and a `near_limit` statistic counting requests above a configurable ratio of the limit, 0.8 by default (the 80% warning described for the customer lifecycle). Every allowed request still costs a Redis update. Measured on the same laptop with the same load generator, the service handled 38,167 checks per second with one Redis update each, against 75,652 for this thesis's leased path (`docs/thesis/evidence/ceiling_and_envoy_2026-10-19.md`).

Unkey and **Upstash Ratelimit** are recent services aimed at API businesses. Unkey implements a sliding window and makes decisions in the region nearest the caller; its documentation describes global convergence within seconds for identifiers approaching their limit, which means a small overshoot across regions is possible [6]. Upstash provides fixed window, sliding window and token bucket algorithms as a client library over its Redis service, and recommends the fixed window for multi-region use because the sliding window then needs many Redis commands per check [7]. Both treat exactness across regions as eventual, and neither separates an exact quota from an approximate burst limit.

Gubernator and **Bucket4j** show that serving checks locally and synchronizing later is a common answer to the cost of a shared store. Gubernator, developed at Mailgun, keeps limits in the memory of a cluster of peers and sends each check to the limit's owner, chosen by consistent hashing, in batches; its `GLOBAL` mode answers on the receiving peer and sends the hits to the owner asynchronously, which its documentation says "allows for greater scale but at the cost of consistency" [20]. Bucket4j, a Java library, offers a `delaying` optimization that serves requests locally until a number of unsynchronized tokens or a time limit is reached, and a `predicting` variant meant to reduce the over-consumption that delaying causes [21]. Neither states a bound on the over-admission across instances; Bucket4j's per-node threshold has the same form as the N × C bound derived in Chapter 3.

Doorman, developed at YouTube, is the closest precedent for the lease half of this design [9]. Clients ask a Doorman server for capacity, receive a share as a time-limited lease that they refresh periodically, and limit themselves locally within it; servers divide capacity according to demand. The approach removes the shared store from the hot path, but it relies on clients to respect their leases. This thesis applies the same idea inside a trusted server-side limiter: the instances that hold leases are the limiter itself, so the leased tokens are enforced, and the over-admission they allow is bounded and measured rather than assumed. The term goes back to Gray and Cheriton, who define a lease as "a contract that gives its holder specified rights over property for a limited period of time" and use the time limit so that host and communication failures cost performance rather than correctness [10].

2.5 Summary

Rate limiting algorithms trade memory for accuracy at window edges; a daily quota is a fixed window because it is what customers buy, and a token bucket is the natural burst limit. Redis makes an exact counter possible through atomic scripts, the Redis clock and hash-tagged keys, at the cost of one round trip per check and of a small window of lost writes on failover. gRPC provides the transport, and its retry semantics argue for reporting a store outage as a decision rather than an error.

The systems in Table 2.1 each sit at one point of the trade-off between exactness and cost: managed gateways and multi-region services accept approximate quotas, and the exact options pay a store round trip on every request. None enforces an exact per-customer quota and serves the high-volume burst check locally with a stated, measured bound on its error. That combination, and its evaluation, is the subject of the following chapters.

Chapter 3. Design

3.1 Requirements

The goals of Section 1.3 say what the service must achieve. This section turns them into requirements that a test or a measurement can check: what the service does (3.1.1), how well it must do it (3.1.2), and the product decisions that no technical argument settles (3.1.3).

3.1.1 Functional requirements

Table 3.1: Functional requirements.

ID	The service must	Designed in
F1	answer `Check{key, rule, cost, request_id}` with a decision and the numbers behind it: allowed, reason, limit, remaining, retry and reset times	3.2
F2	answer up to 1,000 checks in one `CheckBatch` call, each answer in the position of its request	3.2
F3	decide one API call of a customer from one rule, `customer`: suspension, burst limit, then the daily quota under the plan's overage policy	3.4
F4	take each customer's numbers from their plan in Redis, so that a plan change applies on the next call, without a restart, and keeps the day's usage	3.3, 3.4.4
F5	count a call that carries a request id once within five minutes, and give its retries the first answer	3.4.3, 3.6
F6	serve rules not tied to a plan with a token bucket, a fixed window or a sliding window	3.3
F7	follow each plan's failure policy when Redis cannot answer, and mark calls admitted without being counted	3.7
F8	write usage events per customer and minute, the record a billing system reads	3.2
F9	reload its rules file without a restart, and keep the old rules if the new file does not load	3.2.4
F10	report its readiness to a load balancer and its metrics to Prometheus	3.2, 3.7.4

The API separates two kinds of "no". A decision is an answer: the call succeeds, and a reason says what was decided. There are seven reasons (`OK`, `RATE_LIMITED`, `QUOTA_EXHAUSTED`, `OVERAGE`, `SUSPENDED`, `UNAVAILABLE`, `UNVERIFIED`), which the gateway maps to HTTP: 429 [18] for the two limits, with `Retry-After` [17] in whole seconds from the answer's `retry_after_ms`, 403 for a suspended account, 503 for `UNAVAILABLE`. A gRPC error means that no decision could be made: `INVALID_ARGUMENT` for a malformed request (an empty key or rule, a key or request id longer than 128 bytes, a rule the configuration

does not have) and `RESOURCE_EXHAUSTED` for an overloaded instance. A Redis outage is deliberately an answer, not an error. The limiter applies the plan's failure policy and says that it did, so a gateway never has to invent a policy of its own for a failure it cannot see.

3.1.2 Non-functional requirements

Table 3.2: Non-functional requirements, from the goals of Section 1.3, and their final status (Chapter 5).

Goal	Requirement	Verified by	Status
G1	a quota is never over-admitted, whatever the concurrency	400 concurrent calls on a Free quota of 200 through gRPC admit exactly 200; evaluation test 1 with several instances	met: four instances, 20 of 20 runs exact (Free 200, Pay 2,200), 9,608 duplicate requests counted once
G2	a retry with the same request id within five minutes is not counted again	script and gRPC tests (3.4.3)	met
G3	a plan change applies on the next call and keeps the day's usage	mid-day upgrade test; 22 ms from the admin tool to two instances (3.4.4)	met
G4	one reset instant, 00:00 UTC by the Redis clock	day-rollover test of the quota script; evaluation test 2	met: yesterday's full counter resets once although two instances meet it at the same moment (`YesterdaysFullCounterResetsOnceForEveryInstance`); 34,425 rollovers in the property tests
G5	about 1,000,000 checks per second on a cluster	an estimate from per-instance measurements (3.8), never reported as a	estimated in 3.8

Goal	Requirement	Verified by	Status
		measurement	
G6	p50 below 500 µs and p99 below 2 ms per check	latency at fixed arrival rates, measured at the client (an upper bound on the server-side figure)	met by leased rules up to 20,000 checks/s per instance (p50 0.43 ms, p99 1.45 ms; final build, `open_loop_2026-10-24.md`); at 40,000/s p99 3.30 ms. The full customer check meets the p99 at 5,000/s (1.88 ms) but not the p50 (0.94 ms): it needs two Redis round trips
G7	over-admission of a leased limit at most N × C in any window	proof (3.5.3); seeded harness over 100,000 seeds; measurement on the cluster	proved and measured: at most 381.5 tokens against 400 at a 10% cap, a model within 0.3 tokens (3.5.4)
G8	an over-quota flood is refused locally; an overloaded instance refuses at once	unit tests; microbenchmark of the deny cache (3.6.1); flood test	met: a flood of 74,881 denials a second cost 2 Redis calls in 10 s; p99 of a denial 0.61 ms at 10,000/s
G9	documented behaviour per plan when Redis is slow or down	a master killed under load (3.7.3)	met

Two further requirements apply to the code rather than to the service. Nothing on the lease hot path takes a lock, allocates memory or writes a log line, because each of those would cost more than the compare-and-swap that serves the check. And no change is called an optimisation without a measurement before and after it; most sections of this chapter quote one.

3.1.3 Product decisions

Some requirements are not technical consequences but product choices: several answers would be defensible, and the system must pick one and state why. Table 3.3 lists the six that affect how quotas are counted. Each is enforced in one place, so changing a decision later changes one component, not the whole design.

Table 3.3: Product decisions.

Question	Decision	Rejected alternative
When does the day reset?	00:00 UTC for every customer	midnight in the customer's time zone
Does a call that fails with a server error count?	yes	refund the unit on a 5xx response
When does a downgrade apply?	at the end of the billing cycle	immediately
Is traffic admitted while Redis is unreachable billed?	no: it is marked unverified	billed like counted traffic
What if a customer has no plan hash?	treat the customer as Free	deny
How large is the grace allowance?	10% of the daily quota, stored per plan	a fixed amount for every plan

Reset at 00:00 UTC. The quota script derives the current day from the Redis clock as ⌊now / 86,400,000 ms⌋. Every limiter instance therefore agrees on the reset instant (goal G4), the script needs no time-zone table, and no job runs at midnight: the first call of a new day sees a new day number and starts from zero. Per-customer midnight would require a time zone in every plan, would make "today" differ between customers, and would give some customers 23- and 25-hour days at daylight-saving transitions. It is listed as future work.

Server errors count. The limiter decides before the call reaches the provider's service and never sees its outcome. Refunding a unit after a 5xx response would need a second message from the gateway after the response, adding a call per failed request and a new failure mode: a lost refund message silently overcharges, a duplicated one undercharges. The call also did consume the provider's capacity. A refund keyed by `request_id` is possible future work, which the API already supports because every call can carry an id.

Downgrades at the end of the cycle. A customer has paid for the current cycle, and the quota script reads the limit from the plan at the time of each call, so a downgrade is simply written to the plan hash later, by the admin tool, when the cycle ends. If a downgrade does apply on a day whose usage already exceeds the new limit, the script denies with `remaining = 0`, never a negative number (`QuotaDowngradeBelowUsageDeniesWithoutNegativeRemaining`). An immediate downgrade would cut service the customer has already paid for.

Fail-open traffic is not billed. When Redis cannot be asked and a plan fails open, the call is admitted without being counted, and the response says so (`REASON_UNVERIFIED`). Billing such calls would bill traffic that was never measured and might exceed the quota; the exactness promise of this thesis covers counted traffic only. Usage events mark these calls `unverified`, so a billing system can apply its own policy to them.

A missing plan means Free. A plan hash can be missing because a sign-up has not propagated yet, or because of an operator error. Treating the customer as Free can give

away at most the cheapest plan, while denying would turn a data error into an outage and fail every new customer whose first call races the sign-up. The same rule applies to individual fields: a missing or malformed field takes its Free default (`plan_test.cc`).

Grace of 10% of the quota. A single rule is easy to state in a price list and to test: a Pay customer with 2,000 calls per day may make 2,200 before being denied. The amount is stored per plan in the `extra` field of the plan hash, written by the seed tool and the admin tool, so a plan can carry a different allowance without a code change; 10% is the value of the shipped presets.

3.2 Architecture

3.2.1 Components

Figure 3.1 shows the processes that take part in a check: the provider's API gateway, N identical limiter instances, and one Redis Cluster. Only Redis holds state that matters; everything an instance holds can be lost.

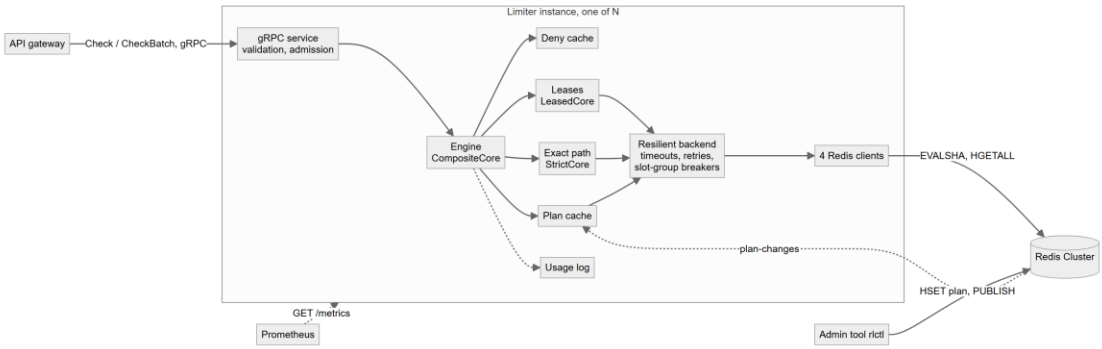


Figure 3.1: Components. Solid arrows are calls on the path of a check; dashed ones are not. (Source: `figures/components.mmd`, Mermaid.)

API gateway. The gateway is outside the system built here; a small demo gateway (`examples/gateway-python`) plays its part in the tests. It authenticates the end user's API key, maps it to a customer id, and asks the limiter before it forwards the call: one `Check` naming the `customer` rule per API call, or a `CheckBatch` of up to 1,000 checks. The limiter trusts the gateway: it sees customer ids, never API keys, validates only the shape of each request, and is not reachable from the internet.

Limiter instances. Every instance runs the same code and configuration and none talks to another. What an instance holds is either a copy of Redis data (cached plans, local denials) or tokens Redis granted it (leases), so adding an instance needs no coordination and losing one costs at most the tokens it held, which makes the limiter briefly stricter. Inside an instance, a check passes four layers:

1. The **gRPC service** validates each request at the trust boundary and admits at most 10,000 checks at once (3.7.4). It uses gRPC's callback API: a check that waits for Redis holds a callback, not a thread.

2. The **engine** decides. `CompositeCore` handles the `customer` rule in the order of 3.4.1; `LeasedCore` serves rules not tied to a plan, and the quota of plans of 100,000 calls a day or more, from leases (3.5); `StrictCore` sends one script per check. The burst limit of a customer is a script per check today: leasing it per customer is future work.
3. **Shared local state**: the plan cache (3.4.4), the deny cache (3.6.1) and the usage log, which sums each customer's outcomes per UTC minute in 16 lock-free rings and appends one JSON line per sum to a file every second, the durable record a billing system would read.
4. **Redis access**: the resilient backend of 3.7.2 (timeouts, retries, breakers per slot group) in front of four redis-plus-plus cluster clients, each with its own I/O thread, keys spread over them by hash. Scripts are sent as `EVALSHA` with a SHA1 computed locally; a node that does not know the script yet answers `NOSCRIPT`, and the call is sent once more with the source.

Redis Cluster. The only shared state and the only source of truth: plans, counters, token buckets and stored answers. Each script runs atomically on the master that owns its keys, and all the keys of one customer carry the customer's hash tag, so one script touches one node and needs one round trip. Scripts read the clock of that node, so every instance agrees on the time and on the UTC day. The development cluster has three masters; the fault tests add a replica to each.

Admin tool. `rlctl` writes a customer's plan hash and then publishes the customer id on the `plan-changes` channel, which every instance subscribes to (3.4.4). It stands in for the provider's account and billing systems, which are out of scope.

3.2.2 The path of one check

Take `Check{key: "cust:42", rule: "customer"}` for a Pay customer.

1. A gRPC thread receives the call. The service validates it and counts it as in flight.
2. The plan comes from the calling thread's own copy (no lock), else from the shared cache, else from one `HGETALL` that every check of this customer waiting at that moment shares.
3. A suspended plan, or a denial in the deny cache, answers at once without Redis.
4. The burst script runs on the master that owns `{cust:42}`. A denial answers `RATE_LIMITED`.
5. The quota script runs on the same master. For a plan of 100,000 a day or more, a lease of the day's counter answers instead, and Redis is asked only to refill it.
6. The Redis I/O thread that received the reply builds the answer, finishes the RPC, records the metrics, and records a usage event.

A Pay or Free customer's check therefore costs two Redis round trips, one after the other (one when the burst limit denies), plus a plan read every five seconds per instance; an enterprise customer's check costs one, plus refills; and a check of a leased rule such as `api_default` costs none when its lease holds tokens, and is then answered on the gRPC thread itself. Both of a customer's keys live on one master, so a single script could check

burst and quota in one round trip. The design instead moves the burst limit towards a per-customer lease, which removes its round trip altogether.

No thread waits for Redis. Table 3.4 lists the threads of an instance.

Table 3.4: Threads of a limiter instance.

Thread	Count	Work
gRPC callback threads	gRPC's pool	receive, validate, admit; answer lease hits and local denials inline
Redis I/O threads	4	send scripts, receive replies, run the engine's callbacks, finish RPCs
Retry timer	1	sends timed-out calls again after their jittered delay (3.7.2)
Plan listener	1	the `plan-changes` subscription
Usage flush	1	sums the usage rings and appends to the usage file, once a second
Metrics	1	answers `GET /metrics`
Main	1	signals, readiness, rule reload, shutdown

3.2.3 Rules and plans

A deployment's rules file says which limits exist and how each is decided; a customer's plan in Redis says how large their limits are (3.3). The split follows how often each changes. Plans change whenever a customer signs up, upgrades or stops paying, so they live in Redis and every instance reads them through the plan cache. Rules change when the provider changes its product, so they live in a file shipped with the deployment.

3.2.4 Reloading the rules

On SIGHUP an instance reads its rules file again without a restart. The signal handler only sets a flag; the main thread, which reads it every 100 ms, parses the whole file first. A file that does not load is logged and changes nothing. Otherwise the main thread builds a new engine from the new rules and switches the gRPC service to it with one atomic pointer store. Checks from then on use the new engine, while a check already inside the old one finishes there. One second later the old engine gives its unused leased tokens back to Redis, as at shutdown, and it is kept until the process exits, so no check can ever reach a freed engine.

Swapping the whole engine, rather than the rule table inside it, follows from the lease table: every lease refers to the rule it was made for, and engine callbacks hold references into the table, so a table that changed under them would leave leases serving old rules. A new engine keeps each table immutable for its whole life. Its costs were measured under an open-loop load of 10,000 checks per second (`docs/thesis/evidence/reload_2026-10-22.md`). Nine reloads, six that applied and three rejected files, failed no call. A pass with three reloads in 24 seconds had a p99 of 1.33 ms, against 0.72 ms without, because the new

engine starts with no leases and each key's first check after the switch waits for one refill. Reading the engine through an atomic pointer cost nothing measurable per check: 63,366 against 63,681 checks per second for the gRPC service alone, before and after the change (medians of eight alternating runs). The memory of a replaced engine, about 8 MB of lease table plus its leases, stays until the process exits; an instance reloaded hundreds of times should be restarted.

3.3 Data model

Three kinds of data decide a check: rules, which are templates in a file; plans, one per customer in Redis; and counters in Redis, which only the scripts write. Instances keep local copies and grants of these (3.3.4).

3.3.1 Rules

Table 3.5: Fields of a rule (`config/rules.yaml`).

Field	Values	Meaning
`algorithm`	`token_bucket`, `fixed_window`, `sliding_window`, `quota`	the Lua script that decides
`limit` or `limit_from`	a number, or `plan.daily` / `plan.burst`	where the limit comes from: the file, or the customer's plan
`refill_per_sec`, `window_ms`	numbers	a token bucket's refill rate; a window's length
`strict`	`true` or `false`	one script per check, or served from a lease
`lease_chunk_pct`	1 to 100, default 10	a leased rule's chunk cap C, in % of the limit: the knob of the bound N × C (3.5.3)
`quota_mode`	`strict` or `auto`	`auto`: plans of 100,000 a day or more lease their quota (3.4.5)
`failure_policy`	`fail_open` or `fail_closed`	when Redis cannot answer, for a rule not tied to a plan
`burst`, `quota`	two rule names	a customer rule: the two rules it combines (3.4.1)

The shipped file has eight rules: `daily_quota`, `burst` and `customer` for customers; `api_default`, a leased token bucket not tied to a plan; `login_attempts`, a strict sliding window; and three small demonstration rules. A file is checked as a whole when it loads, and the first inconsistency names the rule and the field: `plan.daily` only with the quota algorithm, `plan.burst` only with the token bucket, a quota rule must be strict, and a customer rule's two names must exist and be a token bucket and a quota. An instance with a bad file does not start; on a reload it keeps its old rules.

3.3.2 Plans

A plan is a Redis hash, `{cust:42}:plan`, written by the admin tool.

Table 3.6: Fields of a plan hash.

Field	Meaning	Default
`plan`	the plan's name; `suspended` denies every call	`free`
`daily`	daily quota, in cost units	200
`burst`, `refill`	the burst bucket's capacity and refill per second	2, 2
`overage`, `extra`	the policy at the quota (`hard`, `grace`, `cap`) and its amount	`hard`, 0
`failure`	when Redis cannot answer: `open` admits as `UNVERIFIED`, anything else denies	closed

A missing hash, and any missing or malformed field, takes the Free default (3.1.3). The defaults are deliberately the cheapest plan: a broken plan can never give away more than Free. Table 3.7 lists the presets shipped with the admin tool; they are product choices, kept in one file (`tools/plans.sh`).

Table 3.7: Plan presets.

Plan	Daily quota	Burst (capacity, refill/s)	At the quota	Redis down
`free`	200	2, 2	hard: deny until 00:00 UTC	fail closed
`pay`	2,000	20, 20	grace: 10% more free, then deny	fail open
`pay_cap`	2,000	20, 20	cap: up to 1,000 billed overage	fail open
`enterprise`	1,000,000	2,000, 2,000	cap: up to 100,000; quota leased in chunks of 1,000	fail open
`suspended`	0	0	always deny, `SUSPENDED`	deny

3.3.3 Counters in Redis

Table 3.8: Keys in Redis.

Key (example)	Type and fields	Written by	Expires
`{cust:42}:plan`	hash: the fields of Table 3.6	admin and seed tools	never
`{cust:42}:daily_quota:q`	hash: `day`, `used`, `overage_used`	`quota.lua`	end of the UTC day + 1 h
`{cust:42}:burst:tb`	hash: `tokens` (in thousandths),	`token_bucket.lua`	when the

Key (example)	Type and fields	Written by	Expires
	`last_refill_ms`		bucket would be full again, + 1 s
`{cust:42}:daily_quota:req:<id>`	string: the stored answer	`quota.lua`	5 minutes
`{user:42}:demo_fixed_window:fw`	string: the current window's count	`fixed_window.lua`	end of the window
`{user:42}:demo_sliding_window:sw`	hash: the current and previous windows' counts	`sliding_window.lua`	from the window length

Five choices shape these keys.

- **The hash tag is the limited entity.** Redis Cluster places a key by the part in braces only, so every key of one customer is on one master and one script may touch several of them atomically, as the quota script does with the counter and the stored answer. Gateways send the customer id, never a key, so the key format exists in one place in the code.
- **Rule and algorithm in the name.** Several rules of one algorithm can coexist for a customer, and a rule whose algorithm changes starts from fresh state instead of misreading state of another shape.
- **`used`, not `remaining`.** The limit arrives with every call, read from the current plan, so a plan change rewrites no counter (3.4.1).
- **Integers.** A bucket holds thousandths of a token, and a refill adds elapsed milliseconds times the rate, so millions of refills accumulate no rounding error.
- **The Redis clock and a lifetime on every write.** Scripts read the clock of the node that owns the key, so instances with skewed clocks cannot disagree about the day; elapsed time is clamped at zero in case a replica promoted after a failover runs behind. Every key but the plan expires on its own, so Redis memory follows the active customers, not every customer ever seen.

3.3.4 State on an instance

Table 3.9: What an instance holds, and what losing it costs.

State	Size	If the instance crashes
Lease table	2^20 slots (8 MB) and a 192-byte lease per (rule, key) seen; never evicted	granted tokens not yet used are lost: stricter, never looser
Plan cache	one entry per customer seen recently (expired ones are swept as it grows), plus 256 recent plans	read again from Redis

State	Size	If the instance crashes
	per thread	
Deny cache	one entry per customer denied today	the next call asks Redis, which denies too
Usage rings	16 bounded rings, flushed every second	up to one second of usage events

Nothing in the table has to be recovered after a crash: each entry is either derived from Redis or a grant that Redis has already subtracted. The one cost is in the usage file, the record of what each customer used, which can miss the last second of a crashed instance. The lease table is the one structure that grows with the number of keys and never shrinks; evicting idle leases needs safe memory reclamation and is future work.

3.4 Quota and overage logic

3.4.1 One rule per API call

A gateway asks one question per API call: `Check{key: "cust:42", rule: "customer"}`. The `customer` rule names two others, a token bucket for the burst limit and a quota for the daily limit, and the engine reads the customer's plan once and decides in a fixed order:

1. **Suspended plan:** deny with `SUSPENDED`. No script runs, and waiting does not help.
2. **Local denial:** if this instance recently learned that the customer is out of quota, deny with `QUOTA_EXHAUSTED` and the reset time it learned. No script runs (3.4.4).
3. **Burst:** take a token from the burst bucket. A denial answers `RATE_LIMITED` at once.
4. **Daily quota:** count the call against the day under the plan's overage policy (3.4.2). If the quota denies, the burst token taken in step 3 is given back.

The order follows from what each answer costs and means. A suspended account and a known denial cost no Redis call, so they come first. Burst comes before the quota because a call refused for burst must not consume daily quota: the customer did not get the service. The give-back in step 4 is the mirror case. Without it, a customer over quota would drain their own burst bucket with denied calls and later be told `RATE_LIMITED` instead of the true reason, `QUOTA_EXHAUSTED`, which tells the client to wait for the reset rather than retry in a second.

The quota script stores what was used, not what remains, and receives the limit from the current plan on every call (3.3). A plan change therefore rewrites no counter: an upgrade from 200 to 2,000 a day turns 200 used into 1,800 remaining on the next call, and a downgrade below today's usage denies with `remaining = 0`, never a negative number.

3.4.2 Overage policies

What happens at the limit is a product choice, so the plan carries it as a policy and an amount (`overage` and `extra` in the plan hash):

- **hard** denies at the limit until 00:00 UTC.

- **grace** allows `extra` more units free of charge, then denies. The shipped Pay plan has a grace of 10% (3.1.3).
- **cap** allows up to `extra` units of billed overage and marks each such call `OVERAGE`. The script counts overage in a second field, `overage_used`, so the plan's own counter never passes the limit and billing reads the overage directly. A call that straddles the limit (cost 2 with one unit left) bills only the part beyond it.

Request 2,001 of a plan of 2,000 a day shows the difference. Table 3.10 gives the answer of the same call on three customers whose plans differ only in the policy.

Table 3.10: Request 2,001 on three overage policies (2,000 used, plan of 2,000 a day).

Policy	Shipped plan	Answer to request 2,001	State after it
hard	Free	denied, `QUOTA_EXHAUSTED`, retry at 00:00 UTC	used 2,000
grace, extra 200	Pay	allowed, `OK`, remaining 0	used 2,001; denied after 2,200
cap, extra 1,000	Pay with cap	allowed, `OVERAGE`	used 2,000, overage_used 1; denied after 1,000 more

Grace answers `OK`, not `OVERAGE`, because the extra units are free: the reason field tells the gateway what to bill, and only cap overage is billed. That choice is also what gives request 2,001 three different answers, one of the acceptance tests. It is tested on the script alone and end to end through gRPC (`Request2001GivesThreeAnswersOnThreePolicies`). Both counters reset with the UTC day, derived from the Redis clock so every instance agrees on the reset instant.

3.4.3 Retries counted once

A gateway that times out and retries must not charge the customer twice, and must give the client the same answer as the first attempt. A call may carry a `request_id`. The quota script then stores its answer under a key with the customer's hash tag, `{cust:42}:daily_quota:req:<id>`, for five minutes, in the same atomic script call that counts the usage. A retry with the same id gets the stored answer, denials included, and is not counted again. The answer is marked `replayed`, so the retry produces no second usage event and a replayed denial, decided under the plan of the first attempt, is not cached locally under a plan that may have changed since. Storing the answer rather than a "seen" flag matters: a retry of a denied call must be denied too, even if the customer was upgraded in between.

Two limits remain. Burst is not deduplicated: a retry is still traffic, and it takes a burst token. And the local denial of 3.4.4 answers before Redis, so a retry of a call that was allowed, arriving after the quota ran out on that instance, is denied. It is not counted twice, but its answer differs from the original.

3.4.4 Local denials and plan changes

After a `QUOTA_EXHAUSTED` answer the instance remembers the customer until the reset time Redis returned, so further calls are denied without a Redis call and the local entry ends exactly when the Redis counter does. Each entry keeps a stamp of the plan's quota terms (daily limit, policy, extra). A call under a different plan ignores and drops the entry, so an upgrade lifts the denial on the next call even if no plan-change message arrives.

Plans themselves are cached on each instance for five seconds, which removed the plan read from every call: on an early strict benchmark the cache alone raised throughput from 16,738 to 19,480 checks per second (median of five runs each, same session). The admin tool writes a plan, then publishes the customer id on a Pub/Sub channel, and every instance drops its cached plan and its local denial. A read in flight when the message arrives is discarded and sent again, because it may have read the old plan. An instance that loses its subscription drops every cached plan when it reconnects, because Pub/Sub keeps no messages for an absent subscriber. A plan change therefore applies within the delay of the message, measured at 22 ms from starting the admin tool until two instances held the new plan, and within the five-second TTL if a message is lost while the connection looks healthy.

3.4.5 Leased quota for large plans

An exact quota costs one Redis call per check, which is the right price for small plans: one chunk of a 200-a-day quota would be a visible share of the day. For a large plan it is not. A plan of 100,000 calls a day or more therefore takes its daily quota from a lease, with the protocol of 3.5 and a smaller chunk: at most 0.1% of the daily limit, so C = 1,000 for the enterprise plan of 1,000,000. The lease is refilled by the quota script itself, with the chunk as the cost and the hard policy, so a lease only ever takes units inside the plan. When the day's counter has less than a chunk left, the lease takes what is left and then denies locally until the reset; a call it denies goes to the exact path, which applies the plan's overage policy and request id. Overage, which is billed, is therefore always counted exactly.

The bound of 3.5.3 applies with no refill during the day. Redis grants at most `limit` units in a day. Units leased before midnight and used after it were counted for the old day, so N instances admit at most limit + N × C in one day: 0.4% of the day for four instances. Units leased and not used before the reset are an under-admission of at most N × C. In the acceptance test four instances shared one customer of 100,000 a day (C = 100) with 20% more demand than the bound allows. They admitted exactly 100,000 units, with 1,253 quota calls for 12,048 checks instead of one per check. A leased call does not deduplicate request ids: a retry inside the plan counts again and uses up one more unit of the plan, though the retry itself is never billed as overage.

The accuracy of leased quota under faults and over a day boundary was tested by the property tests: over 80,490 simulated days with outages and rollovers, Redis never granted beyond the plan, and N instances admitted what Redis granted minus at most N × C held in leases (5.2).

3.5 Lease protocol and the accuracy bound

3.5.1 Why leases

A strict check costs one Redis round trip, and its throughput on one instance is bounded by how many round trips are in flight: an early baseline admits 10,024 checks per second at a p99 of 10.5 ms with 64 concurrent calls, and needs 1,024 concurrent calls to reach 25,301. The burst limit is checked on every API call, almost always for customers far below their limit, so most of those round trips only confirm that tokens remain. A lease removes them: each limiter instance takes a share of a customer's token bucket from Redis in advance and admits checks against it in its own memory, asking Redis again only when the share runs low. The cost is accuracy: tokens one instance holds are invisible to the others. This section defines the protocol, states the invariant it maintains, and derives the bound on the over-admission that invariant allows.

3.5.2 The protocol

A *lease* is the set of tokens of one (rule, key) bucket that one instance has already taken from Redis. Redis remains the owner of the bucket: it refills the bucket over time with `token_bucket.lua` and grants tokens to leases on request, atomically.

1. **Hit.** A check of cost c takes c tokens from the lease with one compare-and-swap that never lets the count fall below zero. It takes no lock, allocates no memory and sends nothing to Redis.
2. **Single-flight refill.** At most one refill per lease is in flight. A check that finds the lease empty is parked; when the refill lands, the parked checks are answered in arrival order. Five hundred concurrent misses on one key cause exactly one refill (tested).
3. **Early refill.** A hit that leaves less than 30% of the current chunk starts a refill, so a key under steady load never waits for Redis.
4. **Top-up.** A refill asks Redis for `target − tokens held`, where the target is the chunk (or the cost of a parked check, if larger). After it lands, the lease holds at most one chunk.
5. **Large checks.** A check whose cost exceeds the chunk cap C skips the lease and is checked in Redis directly, on the same bucket.
6. **Adaptive chunk.** The chunk starts at the cap $C = \max(1, \lfloor \text{limit} \times p / 100 \rfloor)$, where p is the rule's `lease_chunk_pct` (10 by default), and changes once per full refill: it doubles when checks found the lease dry while the refill was out, because demand outran one chunk within one round trip, and halves when the previous chunk lasted more than ten seconds, because its tokens sat unused where no other instance could use them. It never leaves $[\min(8, C), C]$.
7. **Denial.** When Redis reports the bucket empty, the lease denies locally until the retry time Redis returned, so a client hammering an empty bucket costs no further Redis calls.

3.5.3 Invariant and bound

Invariant. At any moment an instance holds at most C tokens of one bucket, apart from the instant between a refill landing and the parked check that asked for those tokens taking them.

Steps 4 and 5 exist to keep this invariant; 3.5.4 explains how its absence was found.

Theorem. Let N instances serve one bucket of capacity `limit` and refill rate r, each holding at most C tokens. Over any interval [a, b], the tokens admitted satisfy

$$\text{admitted}(a, b) \le \text{limit} + r \cdot (b - a) + N \cdot C.$$

Proof. Every token admitted during [a, b] was either held by some instance at time a, or granted by Redis during [a, b]: leases only hand out tokens that Redis granted. By the invariant, the instances hold at most N·C tokens at a. Redis executes each grant atomically against the bucket, which contains at most `limit` tokens at a and gains r·(b − a) during the interval, so it grants at most limit + r·(b − a). Adding the two bounds gives the result. ■

An exact token bucket admits at most limit + r·(b − a) over the same interval, so the over-admission caused by leasing is at most **N × C in any window**. It is temporal, not cumulative: tokens taken from Redis earlier are spent later, but no token is ever created, so over a whole day the leases admit no more than Redis granted. The tuning rule follows directly: the cap is chosen from the over-admission one can accept. With four instances and the default 10% cap, a burst limit of 1,000 may admit up to 400 extra checks in a window, and a cap of 1% would reduce that to 40 at the cost of ten times more refills. Each rule sets its own cap (`lease_chunk_pct`, 3.3.1); 3.5.4 measures both.

3.5.4 Evidence: a seeded harness

The theorem holds only if the implementation keeps the invariant, which is hard to see in concurrent code. A deterministic harness checks it. One to four engine instances share a fake Redis that executes the arithmetic of `token_bucket.lua` (milli-tokens, a millisecond clock, refill on use) at the moment it answers a call. It answers pending calls in an order drawn from a seeded random generator, fails 2% of them, and runs under a fake clock that advances by milliseconds with occasional jumps of up to three seconds. Each seed draws a limit between 10 and 2,000, a refill rate, and checks of cost 1 to 5 on up to three keys, then verifies the theorem over every window of admissions in one pass (prefix sums of admitted cost, with a running minimum). A failing seed prints itself and replays exactly.

The harness found two violations in the first version of the protocol:

1. The early refill added a full chunk on top of the up to 30% still in the lease, so an instance could hold 1.3 × C. Seed 1177 (N = 2, C = 9) admitted 18.79 tokens over the exact bucket in one window, against a bound of 18. The top-up rule (step 4) fixed it.
2. A check costing more than C made the lease hold more than C, through refills sized for it and through partial grants it could not use. Seed 4388 (N = 2, C = 1, costs up to 5) admitted 2.7 over the exact bucket against a bound of 2. Routing such checks to Redis (step 5) fixed it.

With both fixes, 100,000 seeds (165 million checks) keep the bound, and the worst window reaches exactly N × C: the bound is tight, not merely safe.

The bound was then measured on the cluster (evaluation tests 4 and 12, `docs/thesis/evidence/burst_bound_2026-10-22.md`). Four instances served one hot key of a bucket of 1,000 tokens refilled at 100 a second, at chunk caps of 1%, 5% and 10%, under two demands of 2,000 calls a second. The first was a burst after an idle period, the worst case the proof allows: the leases are full and the bucket has refilled. The second was steady overload. The over-admission was taken from the allowed answers as the largest excess over any exact bucket in any window, and the same calls were replayed through a simulation of the protocol written independently of the engine.

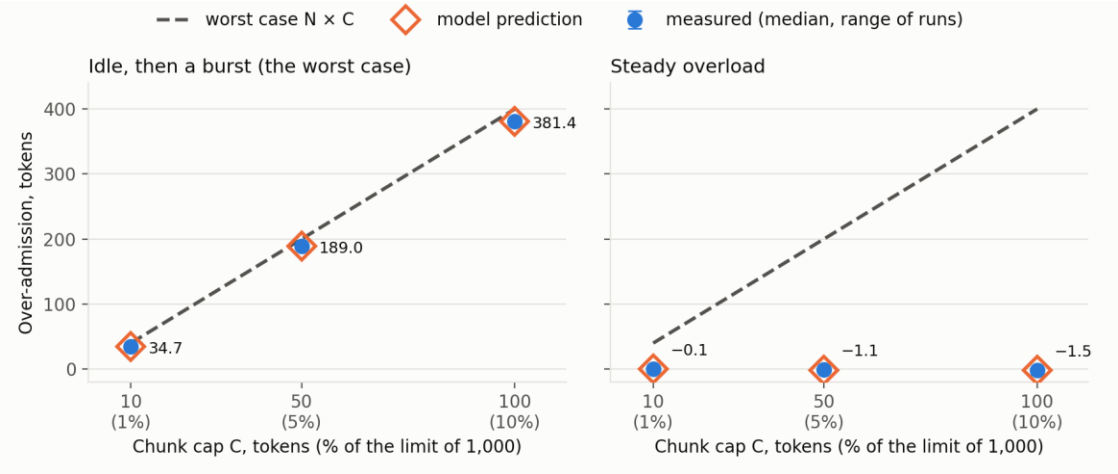


Figure 3.2: Over-admission against the chunk cap, measured (median of five runs) and predicted, with the worst case N × C.

After an idle period the burst admitted 34.7, 189.0 and 381.4 tokens more than any exact bucket could (medians of five runs), against bounds of 40, 200 and 400. Each instance held C − 1 tokens after its warm-up check, so about N × (C − 1) was expected, a little less because part of the bucket's refill lands within the window. Under steady overload the excess was at most zero, and the leases admitted slightly fewer calls than an exact bucket would have. The simulation predicted 34.4, 188.8 and 381.4, and its admitted counts matched the measurement exactly in 29 of 30 runs. The cap is therefore a measured knob: on four instances, a cap of 1% bounds a burst to 35 extra tokens and 10% to 381, at the price of about one Redis call per C checks.

3.5.5 Alternatives considered

- **No lease (strict checks only).** Exact, but one Redis round trip per check: rejected for the burst limit by the throughput argument of 3.5.1, kept for the daily quota where exactness is the product.
- **Adding a full chunk at the watermark.** Simpler, and up to 30% fewer refills than the top-up, but it breaks the invariant (3.5.4). The top-up costs refills of 0.7 to 1.0 × C instead of exactly C, at most 43% more of them. Measured under load (one instance, 192 in flight, 3 runs), the mean top-up was 0.87 of the chunk over Zipf keys

on 10,000 keys, so 14.3 to 14.7% more refills. Over 1,000,000 keys it was 0.994, so 0.6% more, because nearly every refill fills an empty lease (`rl_lease_topup_tokens_total`).

- **`fetch_sub` instead of a compare-and-swap loop.** Measured 2.5 to 2.7 times faster on one shared lease under contention, but a failed take must be undone, and while it is undone another thread can be denied although tokens exist. The compare-and-swap never lets the count go negative. Rejected unless a profile of the real server shows the take on its critical path.
- **Cooperative client-side leases**, as in Doorman [9]. They remove the shared store from the hot path but trust clients to respect their grants. Here the lease holders are the limiter instances themselves, so the tokens are enforced and the bound holds by construction.
- **Starting chunks at the minimum** (8 tokens) and growing them. Only a dry lease doubles a chunk, and with early refills a steadily used lease is never dry, so a moderately busy key would stay near the minimum. Its checks would rarely wait, but one in eight would cost a Redis call. Starting at the cap and halving idle chunks keeps a key at 3 checks per second near 25 tokens: one Redis call per 25 checks.

3.6 Deny cache and idempotency

Sections 3.4.3 and 3.4.4 described what the two mechanisms do for a customer. This section describes how they are built and what they cost, since both sit on the path of every customer check.

3.6.1 Deny cache

A customer over quota who keeps calling, a client stuck in a retry loop for instance, must not cost a Redis call per attempt (goal G8). After the first `QUOTA_EXHAUSTED` answer, an instance therefore answers that customer itself until the reset.

Structure. The cache has 64 shards, each on its own cache line, with a hash map under a mutex and an atomic count of the map's entries. An entry holds the instant the denial ends, computed from the reset time Redis returned, and the stamp of the plan it was made under (3.4.4). A lookup hashes the customer id to a shard and first reads that count. Few customers are over quota at any moment, so almost every lookup finds a shard with no entries and returns after that one atomic load, without the lock; only a shard that holds a denial is locked and searched.

Cost. In the microbenchmark on 16 threads (`results/2026-10-18_customer_microbench`), the lookup that finds no denial runs at 2,089 million lookups per second in total, against 12.2 million when every lookup took the shard's mutex. A flood, every thread looking up one denied customer, contends on one shard's mutex and runs at 9.7 million lookups per second, more than 100 times the gRPC rate of one instance (82,565 calls per second with no limiter at all), so a flood is answered locally without becoming the bottleneck. The flood test confirmed it on the server (evaluation test 5, `docs/thesis/evidence/flood_cardinality_2026-10-22.md`). One customer out of quota, retrying flat out, was denied 74,881 times a second, the instance's rate for any check, and

cost Redis two calls in ten seconds, both of them plan re-reads. At a fixed 10,000 calls a second a denial took 0.24 ms at the median and 0.61 ms at p99. The lock stays.

The cache holds quota denials only. A tenant within its daily quota but far over its burst limit gets `RATE_LIMITED` from the burst script, one Redis call per attempt. In the noisy-neighbour test (evaluation test 11, `docs/thesis/evidence/noisy_neighbour_2026-10-23.md`), one such tenant at 20,000 calls a second raised its master's load from 3,319 to 25,233 calls a second, and the other tenants' p99 from 4.5 to 31 ms; their answers stayed right. The same flood over quota cost Redis nothing. Leased rules already deny locally until the retry time Redis returned (3.5.2, step 7), and the same local denial for the customer's burst check is the fix. It is future work, because the retry time depends on the cost that was denied.

Correctness. A denial recorded at the moment of a lookup in the same shard can be missed once; that check then asks Redis, which denies too, so the cache can make an answer slower but never looser. Instances do not share denials: each learns one from its own `QUOTA_EXHAUSTED` answer, so N instances cost at most N extra quota calls per customer after the quota runs out, and none after that until the reset. Expired entries are dropped when a shard grows past a threshold that doubles with the number of entries left, so the memory follows the customers denied today, not every customer ever denied.

Alternatives. Asking Redis every time is exact and simple, but it multiplies the cost of the round trip by the rate of the flood, which is the cost the cache exists to remove. A deny list shared by the instances would let one denial serve all of them, at the price of messages between instances, which this design otherwise avoids entirely.

3.6.2 Idempotency

Mechanism. A call that carries a request id has its answer stored under `{cust:42}:daily_quota:req:<id>` for five minutes, by the quota script, in the same call that counts it. The key carries the customer's hash tag, so it lives on the same master as the counter, and Redis runs the script atomically: the call is counted and its answer stored, or neither happens. No moment exists at which a retry could find the call counted but no answer stored.

Alternatives rejected.

- *Recording the id first*, with `SET NX`, then counting: two round trips, and a failure between them leaves an id recorded for a call that was never counted, so its retry passes free.
- *Deduplicating in the limiter's memory*: a retry may reach a different instance.
- *Deduplicating in the gateway*: the retry may come from the client behind the gateway, through another gateway instance.

Cost. One stored answer costs 181 bytes of Redis memory: 100,000 answers of the real format, written to one master of the development cluster, raised its memory by 18.1 MB (Redis 7.4.11, 2026-10-22). The memory held is the rate of calls that carry an id, times 300 seconds, times 181 bytes: 543 MB at 10,000 such calls per second, 54 GB at 1,000,000 (3.8).

At that scale the price of counting retries once is memory, not time, and the window is its lever: a gateway's retries of one call typically come within seconds, so a window of one minute would cut the memory by five.

Limits. Only the exact quota deduplicates. The burst limit does not, because a retry is still traffic, and neither does a leased quota (3.4.5). An id reused for another call within five minutes gets the first call's answer, whatever its cost, so ids must be unique per call; the gateway, which generates them, must guarantee that.

3.7 Failure policies and failure modes

3.7.1 What the limiter promises when Redis fails

A Redis outage must not become a limiter outage, nor a storm of retries that keeps Redis down. The limiter therefore answers every check whatever Redis does, and the answer is decided in a fixed order. A lease that still holds tokens keeps serving, because those tokens were granted before the failure. When the lease is empty, or for a check that needs Redis directly, the customer's plan decides: a paid plan fails open and the call is admitted as `UNVERIFIED`, which is neither counted nor billed (product decision of 3.1.3); a Free plan, and any customer whose plan was never read, fails closed with `UNAVAILABLE` and a retry time of one second. The plan cache keeps answering with the last plan it read while Redis cannot be asked, so a customer keeps their policy through an outage longer than the cache's five-second lifetime.

3.7.2 Detecting a failure: timeouts, retries and breakers

Timeouts. Every Redis call has a timeout of 50 ms. The plan asked for 5 to 10 ms, and a 10 ms timeout was tried first; under load it fired while Redis was healthy. With a single Redis client, replies waited tens of milliseconds behind its one I/O thread, which also finished every gRPC call it answered, and a timeout makes the client library drop the whole connection, failing every call pending on it. Four clients with the keys spread over them took the exact path from 8,896 to 36,283 checks per second at 192 calls in flight; with four clients, 50 ms was the smallest timeout that raised no false alarm. A tighter timeout needs the I/O thread freed of finishing calls, which is future work.

The timeout is a deadline on each attempt, which the client library alone did not provide. Its timeout is an idle timeout per connection, re-armed by any reply that arrives. The fault test (5.7) gave one master 80 ms of extra latency under load: its connection was never idle for 50 ms, no call timed out, and that master's customers waited about 250 ms for three slow round trips instead of getting their plan's policy. The resilient layer now answers every attempt as timed out at its deadline, whichever comes first of the reply and the deadline, and drops the late reply. The deadlines wait in a lock-free queue in send order (Vyukov's multi-producer, single-consumer queue [26]), which a timer thread empties and checks in batches. In the whole server its cost is within run-to-run noise, and no deadline was missed at saturation or under open-loop overload at 30,000 checks per second (`docs/thesis/evidence/faults_2026-10-23.md`). A first version guarded the queue with a mutex: in isolation that made each call 448 ns and serialised the threads, and the lock-free queue takes it to 214 ns, with 11 times the throughput at 12 threads

(`docs/thesis/evidence/verification_and_speedups_2026-10-23.md`). With it, a master 80 ms slow, or silent, gives exactly its third of the customers their policy, at a p99 under 4 ms, and recovers within a second.

Retries. A call that timed out may still have run, so only calls whose repetition cannot make a limit looser are sent again: plan reads, and token-bucket scripts, where a repeat at worst takes one more token. Quota scripts are never retried by the limiter. A repeat would count the call twice, and with a request id it would come back as a replayed answer, so the admitted call would never be recorded for billing; a code review of the first version found exactly that. A retried call waits a random 0.5 to 1.5 times an exponentially growing delay, so calls that timed out together do not return together, and is sent at most twice.

Circuit breakers. After 20 consecutive failures a breaker stops calls for 500 ms and then lets a single probe through; the probe's answer closes it or opens it again. An answer that is an error from Redis itself, such as a redirect or a failing script, does not count: Redis answered. The first version used one breaker for the whole cluster, on the reasoning that one dead master among healthy ones could never produce 20 failures in a row. The fault test of 3.7.3 proved the reasoning wrong: when a master dies, every call pending on its connections fails at the same moment, and the breaker opened six times in four seconds for every customer, including those on the two healthy masters. There is now one breaker per group of 256 of Redis Cluster's 16,384 hash slots, computed from each key exactly as Redis computes it, so a dead master opens only the groups of its own slots.

3.7.3 Failover

When a master dies, the cluster promotes its replica after the node timeout, and the client must find the new master. Plan reads go through the client library's routed path, which follows the change. Scripts are sent straight to the node the client's slot map names, so a script that meets a redirect or a lost node is sent once more after a routed command has refreshed the map. A quota script whose connection was lost is the exception, since it may already have run. The Redis Cluster itself must also be configured for partial service: with its default, `cluster-require-full-coverage yes`, every node refuses every command from the moment a master is marked failed until its replica is promoted, which made a third of the slots' outage briefly an outage of all of them.

Table 3.11: A master killed under load (3 runs, open-loop load over 30 s, master killed at 10 s).

Stream	Offered	Answers while its slots were without a master	Recovery
Free customers, `customer` rule	1,000 checks/s	1,095 to 1,442 `UNAVAILABLE`, none admitted	4 to 5 s
Pay customers, `customer` rule	5,000 checks/s	5,417 to 7,113 `UNVERIFIED`, none denied	4 to 5 s
Leased rule `api_default`	10,000 checks/s	leases served for 2 s, then 48 to 519 `UNVERIFIED`	4 to 5 s

During the outage exactly one third of each stream was affected, the share of the dead master's slots, and no call failed. The four to five seconds are the node timeout of two seconds, the promotion, and the first re-routed call.

3.7.4 Overload and shutdown

Backpressure. At most 10,000 checks may wait for an answer at once; beyond that a call is refused immediately with `RESOURCE_EXHAUSTED`. Every queue behind the gRPC service holds admitted checks only, so this one bound limits the memory a Redis stall can consume. Its cost was not measurable. A second bound proved necessary in the open-loop tests: with gRPC's default of unlimited concurrent calls per connection, a load beyond capacity piled up thousands of calls per connection and collapsed even with no limiter in the server, losing most calls at their deadline. At most 100 concurrent calls per connection turned the collapse into queueing that drains (p99 137 ms instead of 2 s at 30,000 calls per second), at the price of a single-connection client on the exact path reaching 22,957 instead of 28,251 checks per second.

Readiness. The gRPC health service reports `NOT_SERVING` while every slot group's breaker is open, that is while this instance reaches no Redis master, and during shutdown, so a load balancer sends the traffic to instances that can reach Redis. A single dead master does not count: every instance sees it, and the plans' failure policies already answer for it.

Shutdown and crash. On SIGTERM the server reports `NOT_SERVING`, drains its calls for up to five seconds, and gives the unused tokens of every token-bucket lease back to Redis before it stops: 72,194 tokens from 1,000 leases in 7 ms in a test, which brought a customer's bucket from 929 tokens back to its capacity of 1,000. A crash loses the leased tokens instead. That makes the limiter briefly stricter, never looser, and needs no recovery.

Table 3.12: Failure modes.

Failure	Detected by	Behaviour	Recovery
Redis master down	refused calls, 50 ms timeouts, slot-group breakers	leases serve; then the plan decides for that master's slots only	replica promoted; next call re-routed; breaker probes close
Redis slow or silent	a 50 ms deadline per attempt	safe calls retried twice with jitter; then as master down	breaker probes, within 1 s
Connections dropped	connection errors	calls in flight at that moment follow the policy	reconnect at once
No master reachable from an instance	every breaker open	health `NOT_SERVING`; answers from leases and plan policies	first successful probe
Missed plan-change message		old plan until the five-second cache lifetime ends	automatic
Limiter overloaded	in-flight count, calls per	`RESOURCE_EXHAUSTED`, or queueing on the connection	the client backs off

Failure	Detected by	Behaviour	Recovery
	connection		
Limiter stopped	SIGTERM	drain, return leased tokens, flush usage events	none needed
Rules file that does not load (SIGHUP)	the parser	logged; the old rules stay live (3.2.4)	fix the file and reload
Limiter crash		leased tokens lost: stricter, never looser; up to one second of usage events lost	none needed

The measurements behind this section are in `docs/thesis/evidence/reliability_2026-10-21.md` and `docs/thesis/evidence/week5_gate_2026-10-21.md`.

3.8 Capacity math

Goal G5 asks for about 1,000,000 checks per second, which is a figure for a cluster of machines. This thesis has one laptop, so this section does what Section 1.3 announced: it takes what one instance and one Redis master were measured to do, and computes how many of each the target would need, with every assumption named. **The results are estimates, not measurements.** The final version, with the scaling measurements, is in Chapter 5 (5.11).

3.8.1 Inputs

Table 3.13: Inputs, measured on one laptop where client, limiter and Redis share 12 hardware threads.

Input	Value	Source (`docs/thesis/evidence/`)
One instance, leased rule, one check per call, 192 calls in flight	89,743 checks/s, p99 4.28 ms (75,652 before the tuning of 5.10)	`verification_and_speedups_2026-10-23.md`
One instance, leased rule, fixed arrival rate: the highest rate tried with p99 below 2 ms	20,000 checks/s, p99 1.80 ms	`week5_gate_2026-10-21.md`
One instance, leased rule, batches of 10	538,605 checks/s, p99 7.51 ms per call	`ceiling_and_envoy_2026-10-19.md`
One instance, exact daily quota (one script per check), 192 in flight	30,376 checks/s, p99 24.2 ms (one run)	`reliability_2026-10-21.md`
One instance,	20,000 checks/s,	`week5_gate_2026-10-21.md`

Input	Value	Source (`docs/thesis/evidence/`)
customer check of Pay customers (two scripts), fixed arrival rate	all answered, p99 19.5 ms	
Script calls one Redis master absorbs	about 120,000/s	`docs/DESIGN.md` section 3 (an early spike: 360,000/s on three masters, each at about one core)
Redis calls per check of a leased rule (refills)	0.016	`throughput_2026-10-19.md`
Redis calls per enterprise customer check	1 script + 0.036 refills	`throughput_2026-10-19.md`
Redis memory per stored request-id answer	181 bytes	3.6.2

3.8.2 Redis masters

The masters needed follow from the Redis calls per check. Table 3.14 takes three traffic mixes, each made of one kind of check only, so that a real mix lies between its rows.

Table 3.14: Redis calls at 1,000,000 checks per second.

All checks are	Redis calls per check	Calls per second	Masters at 120,000 each
a leased rule such as `api_default`	0.016	16,000	1 (an eighth of one)
customer checks, enterprise plans (quota leased)	1.036	1,036,000	9
customer checks, Free and Pay plans	2	2,000,000	17

Plan reads. Each instance reads each active customer's plan once per cache lifetime of five seconds, so plan reads grow with the product of instances and customers: N × A / 5 per second. For 100,000 customers active in every five seconds and 50 instances that is 1,000,000 reads a second, as many as the checks themselves. Two changes would remove most of them. The lifetime only bounds how long a lost plan-change message goes unnoticed (3.4.4), so a lifetime of one minute would cut the reads by twelve. A gateway that sends each customer to one instance, by consistent hashing for instance, would cut them to A / 5 and would also shrink the lease bound for that customer from N × C to C. Neither is built; the cost of a plan read on a master was not measured, and is assumed here to be no lower than a script's. One instance spread over 1,000,000 customers already shows the effect (evaluation test 6): 0.89 plan reads per check, and 26% less throughput than over 10,000 customers.

Memory. Stored answers dominate Redis memory if every call carries an id: 54 GB across the cluster at 1,000,000 calls per second (3.6.2), about 3 GB on each of 17 masters. Plans, counters and buckets grow with the customers, not with the rate: measured, about 420 bytes per customer active today (a plan of 171 bytes, a daily counter and a burst bucket), so 420 MB for 1,000,000 such customers.

3.8.3 Limiter instances

For leased rules the gRPC call, not the limiter, sets the rate of one instance: the leased path ran at 98% of the rate of gRPC with no limiter at all (89,743 against 91,154 checks/s, `verification_and_speedups_2026-10-23.md`). The number of instances then depends on the latency the operator accepts.

- **Throughput:** 1,000,000 / 89,743 ≈ 12 instances, each near saturation with a p99 of about 4 ms.
- **The latency goal G6:** at most 20,000 checks per second per instance keeps the p99 below 2 ms, so 50 instances.
- **Batched calls:** with ten checks per call, 1,000,000 / 538,605 ≈ 2 instances by throughput, with a p99 of 7.5 ms per call.

Customer checks wait for Redis. One instance answered 20,000 customer checks of Pay customers per second at a fixed rate, all of them, with a p99 of 19.5 ms; the rate at which it stops keeping up was not measured. The target therefore needs at most 50 instances for Pay and Free customers, and an enterprise check, with one script instead of two, should need no more. No instance count meets G6 with today's customer check: its p99 was already 2.14 ms at 5,000 checks per second.

Table 3.15: Estimate for 1,000,000 checks per second.

All checks are	Instances	Masters for the checks	What limits it
a leased rule	12 (50 for p99 < 2 ms; 2 with batches of 10)	1	the gRPC call
customer checks, enterprise plans	at most 50	9	the burst script
customer checks, Free and Pay plans	at most 50	17	two scripts per check

The table shows where the design's cost lies: in the customer check's two Redis round trips. Leasing the burst limit per customer, the future work named in 3.2.1, would turn the last row into the second and, for customers far below their limit, much of the second into the first.

3.8.4 Assumptions

Each assumption is a way the estimate can be wrong; Chapter 5 checks those it can.

- **A server instance does what a laptop instance does.** On the laptop the load generator, the limiter and Redis compete for the same 12 hardware threads; a server gives each its own cores, which should help, but no server was measured.
- **Instances scale linearly.** They share nothing but Redis, which makes it plausible. Four instances on the laptop reached 116,515 checks per second, which only shows that the laptop ran out of cores, not how instances scale. The benchmark matrix confirms it: one to four instances on the same laptop share its CPUs and add no capacity, and none of the engines degrades with more instances (5.6).
- **A master absorbs 120,000 script calls per second.** This was measured with the token-bucket script. The quota script does more work (deduplication, overage) and was not measured on its own.
- **Keys spread evenly over the masters.** One customer's keys all live on one master, so a single customer's exact checks can never exceed what one master does: one reason large plans lease their quota.
- **A loopback network.** A real network adds its round trip to every Redis call. Leases hide it from most checks; the exact path and the customer check do not.

3.9 Merchants: one service, many API owners

Sections 3.1 to 3.8 serve one API owner. The service is meant to be sold to many: each API owner (a **merchant**) has its own customers and plans, and must not see or affect another's. This section adds that layer. It changes how a call is named, not how it is decided.

3.9.1 Who is calling

A call carries its merchant's API key in the gRPC metadata `x-api-key`. The service resolves the key to a merchant before any engine work, and refuses a call without a known key with `UNAUTHENTICATED`.

- **Keys are stored hashed.** The hash `{rl:merchants}:keys` maps the SHA-256 of each key to its merchant, so a copy of Redis does not hand out working keys. The admin tool prints a new key once.
- **No lock and no round trip per call.** Each instance loads the hash at start and again on a control message (`!merchants` on `plan-changes`). Each gRPC thread keeps the keys it has resolved, and drops them when a new index is published, so a revoked key stops at its next call.
- **Encrypted on the way.** A key is a bearer credential: whoever reads it can use it. When gateways reach the limiter over a network the platform does not own, the server runs with TLS (a certificate and key in `RL_TLS_CERT` and `RL_TLS_KEY`). Plain text remains for a limiter beside its gateway.

Rejected alternatives:

- *A merchant field in the request:* any caller could claim to be any merchant.
- *A Redis read per call:* a round trip on every call, the cost leases exist to avoid.
- *A TLS client certificate per merchant:* stronger, but every merchant's gateway would then manage certificates. It remains possible later.

3.9.2 Isolation by name

The service turns customer `cust:42` of merchant `acme` into the key `acme|cust:42` before the engine sees it. Every Redis key, plan, lease, cached denial and usage event below the service is built from that key, so merchants share nothing but capacity.

- **Unambiguous:** a merchant id is 1 to 64 of A-Z, a-z, 0-9, _ and -, so it cannot contain |.
- **Spread:** the hash tag stays per customer (3.3.3), so a merchant's customers spread over all masters.
- **No allocation:** the key is built in a buffer each thread reuses, and the engines copy what they keep before the call returns.

Rejected alternatives:

- *A Redis database per merchant:* Redis Cluster has only one database.
- *A cluster per merchant:* the cost would grow with the number of merchants, not with the traffic.

3.9.3 Limits that are not fixed

Each customer's plan hash already holds its own numbers (3.3.2); a plan is a template copied into it.

- **A catalogue per merchant:** a merchant defines its plans with any numbers (`{rl:m:<merchant>}:catalog`); the platform's plans remain the defaults.
- **Custom limits:** any field of one customer's plan can be set for that customer alone.
- **Boosts:** extra units until a given time, for example +500 for a day in an emergency.
- **Monthly quotas:** with `period month`, the quota counts per UTC calendar month.

Every change is written first, then announced on `plan-changes`, so it applies on the next call on every instance. A test measured under one second; the plan change of 3.4.4 took 22 ms. Applying a plan replaces the whole hash, so custom limits and boosts end with it.

The Redis clock decides both when a boost ends and when a month turns, as it decides the daily reset. So instances never disagree. A lease cannot see that clock, so monthly and boosted quotas take the exact path; plain daily plans keep the leased path of 3.4.5.

3.9.4 A paying customer at the limit in an emergency

Answer	Policy	What the customer gets	Built
Block	`hard`	an outage at the worst moment	yes (3.4.2)
A bounded grace	`grace`, e.g. +10%	survives small spikes, free	yes (3.4.2)
Allow now, bill later	`cap` (`postpaid` in the API): counted exactly as billable	no outage; pays for the overage	yes, per plan and per customer (3.9.5),

Answer	Policy	What the customer gets	Built
	overage, up to a spending cap		invoiced monthly (3.9.6)
A boost	granted by the merchant, until a time	no outage, on the merchant's terms	yes (3.9.3)
A prepaid top-up	bought by the customer, for this period	no outage; paid in advance	yes (3.9.5)

The recommended default:

- *Free plans:* block.
- *Paid plans:* a small grace, then billable overage up to a spending cap per quota period (a day on a daily plan, so a month's bill holds at most the cap times its days). The cap is what keeps a faulty client from running up an unbounded bill.

Billing a card later never means storing card numbers. A payment provider keeps the card; the service keeps only the provider's customer id.

3.9.5 Managing the service

A management API (`control/`) changes what the limiters answer, while the limiters answer checks. The two meet only in Redis. Every change is one atomic script on the customer's hash, followed by the plan-change message of 3.4.4. The API's main choices, each against the alternative it rejects:

- **Two credentials per merchant.**
  - *The two keys:* a check key for its gateway's calls to the limiter, and a management key for the API, both stored as hashes.
  - *Why separate:* a gateway that leaks its key cannot change plans.
  - *Isolation:* the merchant always comes from the credential, never from the URL.
  - *Rejected:* one key for both; or the merchant in the path, checked against the key, where a single missing check would open another merchant's customers.
- **Top-ups belong to a period.**
  - *How:* the plan hash holds the prepaid units and the period they were bought for. The script adds them only in that period, by the Redis clock, so they end without a cleanup job.
  - *What survives:* moving a customer to another plan keeps them, since they were paid for.
  - *Retries:* a top-up request may carry an idempotency key, so a client retrying after a timeout does not buy twice. Each purchase is also recorded for the invoice.
  - *Rejected:* a separate counter of prepaid units, which a plan change or a new period would have to reset in a second place.

- **Alerts come from the quota script.**
  - *How:* the script is the one place that sees a threshold being crossed atomically. It publishes the first time in a period that a customer reaches 80% of its limit in force, reaches the limit, goes into billed overage, or is denied. A bitmask in the counter makes each alert once a period, and a replayed answer sends none.
  - *Delivery:* a separate notifier sends each alert to the merchant's webhook, signed, and retries on errors, so nothing is added to the limiter's request path.
  - *The cost:* delivery is at most once while the notifier is down, because Pub/Sub keeps no messages. The same state stays readable through the API.
  - *Rejected:* a scan of counters for crossings, which grows with the number of customers and reports late.
- **The platform calls only public addresses.**
  - *Why:* a webhook URL is chosen by the merchant, so it could point the platform at its own internal network.
  - *How:* URLs that resolve to private, loopback, link-local or reserved addresses are refused, when registered and again before every delivery, and redirects are not followed. A delivery connects to the very address that was checked, with the name kept for the `Host` header and the certificate, so a name that answers a public address first and a private one next (DNS rebinding) cannot redirect it.
- **Hardening.**
  - *Keys are rotated without a gap:* a merchant replaces its own management key in one step, and the old one stops at once. The operator revokes check keys one at a time, so a gateway moves to a new key before the old one goes.
  - *Guessing is limited:* after 20 failed authentications in a minute, the client's address is refused with `429`. The limit is checked before the key is looked at.
  - *Every change is in an audit log:* who made it (the merchant, the operator or the admin tool), when, and the request with its secrets masked. It is kept in a stream per merchant, written in one place, a middleware, so no endpoint can be left out.

3.9.6 Metering and invoices

A merchant bills its customers for what the service admitted, so the numbers on an invoice must be the numbers the quota decided with. The service keeps them where they are made.

- **Counted in the decision.** The quota script (3.4.2) also keeps the calendar month's totals in the customer's counter, in the same atomic step as the decision: units within the plan (free grace included) and billed overage units. This holds for a daily quota too, since invoices are monthly.
- **Kept until invoiced.**
  - *Filing:* when a new month begins, the script files the last month's totals, if it had billed overage, as a field of its own.

  - *Retention:* a counter holding billed overage is kept 35 days past its month, so the billing job finds it even if the customer never calls again.
  - *Cleanup:* the billing job removes the filed month once it is invoiced.
  - *What stays as before:* a counter with nothing to bill expires after its period. So a flood of unknown customer ids costs no more memory than before.
- **One invoice per customer and month.**
  - *Contents:* the plan's monthly fee, the billed overage at the plan's price, and each top-up bought in the month at the price recorded when it was bought.
  - *Amounts:* integers in cents, each line rounded half up once.
  - *Once:* the invoice id names the customer and the month, so closing a month again writes nothing twice.
- **Charged without a card number.**
  - *How:* the card is typed into the payment provider's form, which returns a token. The service keeps only the provider's customer id and charges it when the month closes, with the invoice as the idempotency key.
  - *Outcomes:* a declined charge is tried again after 1, 3 and 7 days, each try with its own idempotency key. Only when the last one fails is the customer suspended (the `suspended` plan of 3.3.2). A charge the provider settles later stays pending until the provider's signed webhook says how it ended. A customer without a card keeps an open invoice for the merchant to collect.
  - *Not stored:* a card number sent to the API is refused.
  - *The provider:* a sandbox here, which takes the test tokens of a real provider.
- **Months from the subscription date,** if the merchant chooses: a customer's month then starts on the day it joined. The day is a field of its plan hash, kept when the plan changes, and the script counts the monthly quota and the billing month from it. A billing month never moves back, so a filed month is never reopened and overwritten.
- **Proration.** A change of fee during the month is recorded with its time, and the invoice charges each fee for the part of the month it was in force.
- **Persistence.** Redis appends every write to a file and syncs it once a second, so a restarted node keeps its counters, less at most the last second. Replication stays asynchronous, so a killed master can still lose writes it had not passed on; the evaluation measures this rather than hiding it (Chapter 5).

Rejected alternatives:

- *Invoices from usage events* (3.3.4): exact enough for reports, but best effort, written per instance and dropped when a buffer is full. Reconciling them against the counters is kept as a check.
- *A billing database written on every call:* a second write on the request path, and a second record that could disagree with the decision.
- *Waiting for a replica on every quota write (`WAIT`):* closes the failover gap, but adds a replication round trip to every exact check.

3.9.7 Protecting the merchant's backend

A quota is a count per day or month; a backend fails on calls per second. Quotas can be sold beyond what a backend takes at once, because not every customer uses its quota at the same moment.

- **A capacity per merchant.**
  - *The setting:* the merchant sets how many calls a second its backend takes from all its customers.
  - *Where it is taken:* after the customer's own burst and before its quota, so a call it refuses is never counted or billed. While Redis says the capacity is out, a call is refused before the burst check too, from the lease alone. Otherwise a flood over the capacity would cost Redis a burst script and a token return per refused call, which the evaluation found slowing other merchants (5.12).
  - *How:* a token bucket keyed by the merchant, served from leases (3.5) like any other bucket. One merchant's capacity is one hot key, the case leases serve best. N instances admit at most the capacity plus N times one lease chunk.
- **A reserve for paying customers.**
  - *The split:* a share of the capacity (for example 20%) is a second bucket, which only plans with a fee may use.
  - *Under a flood:* Free calls exhaust the first bucket and are refused, while paying customers still get the reserve.
  - *Measured:* a property test floods Free calls at three times the capacity over 2 to 4 instances, and checks every window from 1 ms to 4 s against the bound. Paying customers asking for 70% of the reserve kept every call.
- **A check rate per merchant.**
  - *Why:* the platform protects itself too. The operator sets how many checks a second it takes from each merchant, so one merchant's flood costs the others nothing.
  - *Where it is taken:* first, before any other step, for any rule.
  - *The cost:* a lease hit is one atomic instruction, with no allocation.
- **Endpoint weights** need no new mechanism. A gateway already sends a cost per check (F1, 3.1.1), so an expensive endpoint takes more of the capacity.

Rejected alternatives:

- *An exact check of the capacity on every call:* one hot key per merchant on one master serialises every call of that merchant (3.5.1).
- *A priority queue in the limiter:* queueing would hold calls the limiter must answer at once; the reserve gives the same guarantee with a refusal instead of a wait.
- *A reserve taken from the same bucket by a threshold* (Free refused below 20% of tokens): a lease cannot see how many tokens the shared bucket holds, so the threshold would leak by N × C.

3.9.8 The merchant portal

A merchant runs its business from one page, served by the management API at `/portal`. It has no state of its own: every value it shows is read from the API, and every button is one API call, so the portal can do exactly what the API allows and nothing more.

- **What a merchant sees.**
  - *The month:* calls admitted and billed overage, summed from the quota counters (3.9.6). The counters are where every instance's calls are counted, so the numbers cover all instances. Two readings a few seconds apart give the calls a second, timed by the Redis clock.
  - *Customers:* a page at a time, each with its use of the period as a bar, its billed overage and the alerts sent. The customers of a merchant are listed from an index kept in the merchant's slot, so a page never scans the cluster.
  - *One customer:* its terms, its boost and top-up, and the invoice so far this month, made by the same function that closes a month (3.9.6).
  - *Actions:* move to a plan, set custom limits, grant or end a boost, add a top-up, suspend and resume, define plans, set the capacity and its reserve (3.9.7), read a month's invoices and download them, set the webhook.
- **Signed in with the management key,** sent as a bearer token to the same server and kept only in the browser tab's session storage. The API decides who the merchant is from the key alone (3.9.5).
- **Built against the usual attacks on a web page.**
  - *A strict Content Security Policy:* the page may load scripts, styles and data only from its own server, may not be framed, and runs no inline script. Code injected into the page through a customer id or a plan name would not run.
  - *Text, never markup:* every value from the server is written into the page as text, so a customer named `<img onerror=…>` is shown, not run.
  - *Retries buy once:* a top-up from the page carries a new idempotency key, and a button is disabled while its call runs, so a double click does not buy twice.

Rejected alternatives:

- *A separate web application:* a second deployment and a build step, for a page that only calls the API.
- *A session cookie:* the browser would send it on requests that other sites start, which needs protection against cross-site request forgery. A bearer token is sent only by the page's own code.
- *A live count of refusals:* the counters hold admitted calls, which is what is billed. Counting refusals per merchant would add work to every refused call on the limiter, the calls a flood makes. The portal shows admitted calls, and a customer's first refusal at its quota in a period reaches the merchant as an alert; refusals for capacity are not shown.

Chapter 4. Implementation

The design of Chapter 3 is implemented in C++20: 4,705 lines in `src/` and `include/`, 379 lines of Lua for the Redis scripts, and 6,786 lines of tests. The control plane for merchants (4.7) adds 1,313 lines of Python, 629 lines of the portal's page, script and style sheet, and 1,167 lines of tests. This chapter shows how the parts that carry the thesis claims are built, with short excerpts and the measurements that decided them. The full code, every script that produced a number, and the raw results are in the repository.

4.1 Project structure and build

The code is split into layers that depend only on the interfaces below them (`AGENTS.md`, rule 1). The engines see Redis through one abstract class, `RedisBackend`, with three operations: run a rule's script, give tokens back, and read a plan hash. A test can therefore put a fake Redis, a transcription of the scripts on a fake clock, or the real cluster behind the same engine.

Table 4.1: Source layout.

Directory	Contents
`include/ratelimiter/`	the public interface: `RateLimiterCore`, `Rule`, `CheckRequest`, `CheckResult`, the clock type
`src/core/`	the engines: `StrictCore` (one script per check), `LeasedCore` (leases), `CompositeCore` (the customer rule), `AllowAllCore` (no limiter, for measurement)
`src/lease/`	the lease table, the chunk policy and the deny cache
`src/plan/`	plans and the plan cache
`src/redis/`	the Redis client wrapper, sharding over clients, the batcher, the plan-change listener, key names
`src/reliability/`	the circuit breaker, breakers per slot group, the resilient backend (deadlines, retries)
`src/server/`	the gRPC service, request conversion, `main`
`src/metrics/`, `src/usage/`, `src/config/`	Prometheus text, usage events, the rules file
`lua/`	the four rule scripts and the token return
`tests/`, `bench/`, `tools/`	tests, load generators and benchmarks, admin and report tools

The build uses CMake with five presets: `release` (`-O3 -march=native`), `debug`, `asan`, `tsan` and `coverage`. `scripts/test.sh <preset>` builds a preset and runs every test. The libraries are Ubuntu 24.04 packages except redis-plus-plus, which is built from source:

Table 4.2: Libraries and their versions.

Library	Version	Used for

Library	Version	Used for
gRPC, Protocol Buffers	1.51.1, 3.21.12	the service and its messages
redis-plus-plus over hiredis	built from source, hiredis 1.2.0	the asynchronous Redis Cluster client
abseil	20220623 (through gRPC)	gRPC's mutexes; one setting changed (4.5)
yaml-cpp, spdlog, OpenSSL	0.8.0, 1.12.0, 3	the rules file, logging, SHA1 of the scripts
GoogleTest, Google Benchmark	system packages	tests and microbenchmarks

4.2 Lua scripts

Each rule's decision runs as one Lua script in Redis. Redis runs a script to completion before any other command, so the read, the decision and the write are atomic, and two limiter instances can never both take the last token (Chapter 2, 2.2.1). Three choices recur in every script.

- **Time from Redis.** A script reads `TIME` on the node that owns the key, so every instance agrees on the time and on the UTC day, and a limiter with a skewed clock cannot corrupt state. Elapsed time is clamped at zero, for a replica promoted with a clock behind.
- **Integers.** The token bucket stores thousandths of a token, and a refill adds elapsed milliseconds times the rate. Floating-point drift over millions of refills cannot arise.
- **A lifetime on every write.** An idle bucket expires once it would be full again, and a daily counter an hour after its day ends, so Redis memory follows the active customers.

Listing 4.1: The decision of `Lua/token_bucket.Lua` (milli-tokens).

```
tokens = math.min(cap_m, tokens + elapsed * rate)
local need = requested * 1000
if need <= tokens then
  tokens = tokens - need
  allowed = 1
elseif need <= cap_m and rate > 0 then
  retry_after = math.ceil((need - tokens) / rate)
end
```

`requested` is 1 for a direct check and a whole chunk for a lease refill, so one script serves both.

Listing 4.2: Request ids and the overage policies in `Lua/quota.Lua`.

```
if KEYS[2] then  -- a retry of a request already decided gets the stored
answer
  local stored = redis.call('GET', KEYS[2])
```

```
  if stored then
    local a, u, r, o = string.match(stored, '^(%d+),(%d+),(%d+),(%d+)$')
    return { tonumber(a), tonumber(u), tonumber(r), reset_after, tonumber(o),
1 }
  end
end
-- ... a new UTC day starts the counters at zero ...
if used + cost <= limit then used = used + cost; allowed = 1
elseif policy == 'grace' then ...            -- up to limit + extra, free
elseif policy == 'cap' then ...              -- the part beyond the limit,
billed, up to extra
end
redis.call('SET', KEYS[2], allowed .. ',' .. used .. ',' .. remaining .. ','
.. overage, 'PX', 300000)
```

The stored answer and the counter carry the same hash tag, so they live on one master and are written by the same atomic script (3.6.2). The scripts are tested against a real Redis by 33 tests (`tests/lua_scripts_test.cc`), including concurrency, clock jumps and day rollovers.

The client sends `EVALSHA` with a SHA1 computed locally (OpenSSL), so no `SCRIPT LOAD` round trip is needed. A node that does not know the script, after a restart or a failover, answers `NOSCRIPT`, and the call is sent once more as `EVAL` with the source.

4.3 Redis client and batching

Client choice. The backend uses redis-plus-plus's `AsyncRedisCluster`. An early spike reached 290,000 script calls a second from one thread on one connection per master, and about 360,000 on three masters, where each master used about one core (`docs/DESIGN.md`, section 3). Pipelining came free: calls issued without waiting leave together on the connection, and the pool size made no difference. The library's cluster client offers callbacks for ordinary commands but only futures for `EVALSHA`, so scripts go through `cluster.redis(key)`, a handle to the node that owns the key's slot, which has callbacks. This handle does not follow a failover by itself. A script that meets `MOVED` or a lost node is therefore sent once more after a routed command refreshes the slot map, except a quota script after a lost connection, which may already have run (3.7.3).

Several clients. One redis-plus-plus client has one I/O thread, and that thread also runs every callback, finishing the gRPC calls it answers. At 192 calls in flight on the exact path it was the limit: 8,896 checks per second. `ShardedBackend` spreads keys over four clients by hash, so a key's calls stay in order on one connection, and the exact path reached 36,283 checks per second (`docs/thesis/evidence/reliability_2026-10-21.md`). Eight clients did worse than four on 12 hardware threads.

The batcher, measured and left off. A batcher was built that collects calls and sends them together, on a timer or adaptively. It gained nothing:

Table 4.3: Batcher modes on the exact path (3 runs, `results/2026-10-18_batcher/NOTES.md`).

Mode	p50 at 1,000 checks/s	Checks/s at 192 in flight
off	0.450 ms	11,625
timer, 200 µs	0.582 ms	11,402
adaptive	0.438 ms	11,147

The fixed timer cost 130 µs at the median, and neither mode raised throughput. Calls issued together already leave in one write, and throughput under load was bound by round-trip time times concurrency, not by writes. The batcher stays available (`RL_BATCHER`) for the ablation of Chapter 5.

4.4 The lease table

Layout. A lease is one cache line (`alignas(64)`), so two hot leases used from different cores never share a line. The fields touched by every check come first and are atomics; the slow path's fields follow.

Listing 4.3: A lease (`src/lease/lease_table.h`).

```
struct alignas(64) Lease {
  std::atomic<std::int64_t> tokens{0};
  std::atomic<std::int64_t> deny_until_ns{0};  // Redis said no until then
  std::atomic<bool> refill_in_flight{false};   // single flight: one refill
per lease at a time
  std::atomic<std::uint32_t> chunk;            // written only by the refill
owner
  const std::uint64_t id;                      // 64-bit hash of (rule, key),
compared first
  const Rule& rule;
  const std::string key;
  std::mutex mu;                               // slow path only: parked
checks
  std::vector<Waiter> waiters;
};
```

The table is open addressing over a fixed array of atomic pointers. A slot goes from empty to a lease exactly once, by compare-and-swap, and is never cleared or moved, so a lease pointer stays valid for the table's life and lookups take no lock. A lookup compares the 64-bit id first and reads the key string only on a match. The price is that leases are never evicted, a known limitation (6.2): one instance used 224 MB for about 650,000 leases over 1,000,000 keys (`flood_cardinality_2026-10-22.md`).

The hit. A check that finds tokens takes them with one compare-and-swap that never lets the count go below zero.

Listing 4.4: `TryTake`.

```
inline std::int64_t TryTake(Lease& l, std::uint32_t cost) {
  std::int64_t t = l.tokens.load(std::memory_order_relaxed);
  while (t >= cost) {
```

```
    if (l.tokens.compare_exchange_weak(t, t - cost,
std::memory_order_relaxed)) return t - cost;
  }
  return -1;
}
```

Relaxed ordering suffices because the counter is the only data a hit reads; nothing else is published through it. The tokens a refill adds are published by the refill path, under the lease's mutex, which a check that found the lease empty takes too. A `fetch_sub` with an undo on failure was 1.8 to 2.7 times faster on one shared lease under contention (192 against 109 million operations a second on one thread), but while the undo is pending another thread can be denied although tokens exist. The CAS loop was kept. `alignas(64)` made per-thread counters 14 times faster at 16 threads (1,313 against 93.5 million operations a second; `results/2026-10-18_lease_microbench/NOTES.md`).

Single flight. A check that finds the lease empty parks under the mutex and calls `StartRefill`, which sends a refill only if it wins the compare-and-swap of `refill_in_flight` (acquire-release, so the winner sees the lease as the last refill left it). Five hundred concurrent misses on one key cause one refill (tested). A refill asks for the chunk minus what the lease still holds (top-up, 3.5.2, step 4), so an instance never holds more than one chunk, which the bound N × C requires. When the refill lands, the parked checks are answered in arrival order outside the lock, because their callbacks finish gRPC calls.

The hot path stays free of locks, allocation and logging (`CLAUDE.md`). Measured on the whole engine, a hit over 10,000 keys runs at 23.9 million checks a second on one thread and 151.9 million on 16, with no allocation in the timed loop.

4.5 The gRPC server

Callback API. The service uses gRPC's callback API: a handler starts the engine and returns at once, and whichever thread answers (a gRPC thread on a lease hit, a Redis I/O thread otherwise) calls the reactor's `Finish`. No thread waits for Redis, and each call's `Finish` runs exactly once:

- for `Check`, from the engine's callback;
- for `CheckBatch`, from the callback of the last of its checks to answer. A shared state counts the checks still pending with an atomic counter, and each answer lands in its own response slot.

At the trust boundary, before any engine work, the service rejects with `INVALID_ARGUMENT`:

- an empty key or rule;
- a key or request id longer than 128 bytes;
- a batch of more than 1,000 checks.

A Redis outage is an answer (a reason in an OK response), never an RPC error (3.1.1).

Backpressure is one atomic counter of checks in flight. A call that would take it past 10,000 is refused at once with `RESOURCE_EXHAUSTED`, and a batch larger than the limit itself is invalid, since it could never be admitted. gRPC's own limit of concurrent calls per connection is set to 100 (`RL_MAX_STREAMS`). Without it, an open-loop load past capacity piled up thousands of calls per connection and collapsed even with no limiter in the server (3.7.4).

Engine swap and shutdown. The service holds the engine through an atomic pointer.

- **Reload:** a rules reload builds a new engine and stores the pointer once (3.2.4).
- **Shutdown, in order:**
    1. set health to `NOT_SERVING`;
    2. drain calls for up to 5 s;
    3. return unused leased tokens;
    4. stop retries and answer attempts still waiting;
    5. stop the Redis clients while the engine is still alive, since an early refill may still be in flight.

A one-line speed-up. A profile of the server under load (`perf`, 2026-10-23) showed abseil's mutex deadlock detection at work in gRPC's own threads: `GraphCycles::GetId` alone took 2.7% of the CPU. Ubuntu's abseil checks every lock for cycles. `main` now switches the check off, which raised the leased rule from 72,097 to 89,743 checks per second and the customer rule from 36,469 to 41,654 (medians of 5 alternating runs; `verification_and_speedups_2026-10-23.md`).

4.6 Reliability

Circuit breaker. The breaker is a lock-free state machine: closed, open, half-open, held in one atomic, with an atomic failure count and an atomic opening time. While closed, a success only reads the count, so a healthy breaker writes no shared state per call. After 20 failures in a row it opens for 500 ms. Then one compare-and-swap from open to half-open lets exactly one probe through, whose answer closes the breaker or opens it again. A late success from before the opening cannot close it. There is one breaker per group of 256 of Redis Cluster's 16,384 slots, computed from each key with Redis's own CRC16 and hash-tag rule, so a dead master stops only its own customers' calls (3.7.2).

Retries are only for timeouts, and only for calls whose repetition cannot loosen a limit: plan reads and token-bucket scripts, never quota scripts. They wait a jittered, doubling delay on one timer thread and are counted against the breaker.

Deadlines. Each attempt is answered as timed out after 50 ms unless Redis answered first. The client library's own timeout is an idle timeout per connection, re-armed by any reply, so it did not bound a call: the fault test (5.7) found a slow master never timed out (3.7.2). Every Redis call registers a deadline, so the structure that holds them is on a busy path, and a first version guarded by a mutex cost 448 ns a call and serialised the threads. The final version is Vyukov's intrusive multi-producer, single-consumer queue [26].

Listing 4.5: Adding an attempt to the deadline queue: one atomic exchange.

```
void ResilientBackend::Push(Pending* p) {
  p->next.store(nullptr, std::memory_order_relaxed);
  Pending* prev = head_.exchange(p, std::memory_order_seq_cst);
  prev->next.store(p, std::memory_order_release);
}
```

- **The exchange** makes the attempt the new head. Linking the previous head to it publishes the attempt to the timer thread, which reads `next` with acquire.
- **Why the exchange is sequentially consistent.** When the timer has nothing to watch, it sets an `idle` flag and re-checks the queue before sleeping. A producer adds its attempt, then reads the flag. With both sides sequentially consistent, either the producer sees the flag and wakes the timer, or the timer's re-check sees the attempt; neither can miss the other.
- **Answering exactly once.** The attempt carries an atomic flag claimed by whichever comes first, the reply or the deadline.
- **Results.** The queue made a call 214 ns and gave 11 times the throughput at 12 threads; in the whole server the deadline's cost is within noise. A stress test of 24,000 attempts from 8 threads, answered at once, late or never, gets each answered exactly once, clean under TSan.

Health and metrics.

- **Readiness:** gRPC's health service reports `NOT_SERVING` while every slot group's breaker is open.
- **Metrics:** Prometheus text, written by the server itself because no client library was installed:
  - counters are striped over 16 cache lines, so adding to one is an uncontended atomic;
  - histograms have fixed bounds from 50 µs to 1 s;
  - a rule reload keeps every series and adds the new rules' series behind an atomic pointer to an immutable index.

The measured overhead of metrics was within noise.

4.7 Merchants and the control plane

Section 3.9 adds merchants. On the limiter this is a layer in the service and two steps in the composite engine; beside it, a control plane in Python manages merchants, customers and invoices through Redis alone.

Resolving the API key (`src/server/merchants.h`). The service hands the `x-api-key` metadata to `MerchantKeys::Resolve` before any engine work.

- *The common case costs a hash-map lookup:* each gRPC thread keeps the keys it has resolved in a `thread_local` map, valid while a generation counter is unchanged. A

reload publishes a new index and bumps the counter, so every thread drops its keys, and a revoked key stops at its next call.

- *A key the thread has not seen* is hashed with SHA-256 and looked up in the shared index, an immutable snapshot behind `std::atomic<std::shared_ptr>`. A reload builds the next snapshot off the request path and swaps it in.
- *A wrong key is not remembered,* so a stream of random keys cannot fill a thread's map. Each one costs a hash and a lookup, then `UNAUTHENTICATED`.
- *Reloads that overlap* carry tickets; the last one started wins, so a reload retried late cannot bring back a revoked key.

The merchant's limits. A merchant's capacity, its reserve and the operator's check rate become token-bucket rules that `CompositeCore` takes through the lease table (3.5), keyed by the merchant.

- *Rules live for good:* a lease points at its rule, so a reload never frees one. Merchants with the same numbers share a rule (their keys keep their buckets apart), and a reload with unchanged numbers creates nothing.
- *The order of a check:* the check rate first, for any rule. Then, for the customer rule: suspended, the deny cache, the burst limit, the capacity and its reserve, the quota. A refusal for capacity gives the burst token back, so it is neither counted nor billed.
- *A flood over the capacity is refused early:* `LeasedCore::DeniedLocal` reads, without taking a token, whether Redis said recently that a lease's bucket is empty. While it says so for the capacity (and the reserve, for a plan with a fee), the check is refused before the burst script, at the cost of one lease lookup.
- *When Redis cannot answer* for a merchant's bucket, the bucket admits. The customer's own limits, checked around it, still apply with their failure policy (3.7).

Metering in the script (`lua/quota.lua`). The quota script keeps, in the same counter and the same atomic step as the decision, the calendar month's units within the plan and its billed overage. When a new month begins it files the old totals under `closed:m<month>`, if they hold billed overage, and keeps the counter 35 days past its month. The property tests' transcription and model (4.8) were extended with the same rules.

The control plane (`control/rlcontrol/`, Python 3.12) is a separate process. It shares nothing with the limiter but Redis and the `plan-changes` channel.

Table 4.4: The control plane.

Module	Lines	Does
`store.py`	181	every key name and plan field the Python side uses, mirroring `keys.h` and `plan.h`; the atomic scripts that change a customer
`api.py`	558	the management API (FastAPI): operators create merchants and keys; merchants manage plans, customers, boosts, top-ups, capacity, webhooks and invoices
`billing.py`	241	closing a month into invoices, previews, CSV export, reconciliation with the usage events

Module	Lines	Does
`payments.py`	82	a payment sandbox: provider tokens only; a card number is refused
`notifier.py`, `webhooks.py`	200	alerts from `rl-alerts` to the merchants' webhooks, signed with HMAC-SHA256, to checked public addresses only
`periods.py`	49	calendar months by the Redis clock, as the script counts them
`portal/`	629	the merchant portal: one HTML page, its script and its style sheet

- *One write per change:* every change to a customer is one Lua script on its plan hash, then its name on `plan-changes`, the same contract as the admin tool `tools/rlctl`. A script refuses to create a customer that exists or to change one that does not, so two operators cannot interleave.
- *The merchant comes from the credential:* the API resolves the management key's SHA-256 to its merchant and builds every key from that, never from the URL.
- *Invoices are made once:* the invoice id is the customer and the month, and a charge sends it to the payment provider as its idempotency key.
- *A spreadsheet cannot be attacked through the CSV export:* a cell starting with `=`, `+`, `-` or `@` is escaped.
- *Dependencies are pinned:* every package and its dependencies to an exact version, and the installer to a SHA-256.

The portal is three static files served by the API from a fixed list, under a Content Security Policy that allows nothing but its own server. The script, without a framework or a build step, writes every value with `textContent`. It is tested end to end in headless Chrome: a script signs in, uses every action, checks what the page then shows, and fails on any script error (`tests/eval/portal_ui.mjs`, 21 checks).

4.8 Testing

Table 4.5: Tests by kind (281 C++ tests, every one passing in release, ASan and TSan; 48 of the control plane).

Kind	Count	Against	Examples
Unit	217	fakes on a fake clock	each branch of the composite engine, lease protocol, plan cache races, breaker states, deadlines; the merchant key index, a merchant's check rate, capacity and reserve
Script	43	a real Redis (port 6380)	each algorithm's limits, request ids, overage, clock jumps, day rollover; months, boosts, top-ups, alerts, billing totals
Integration	21	the three-master cluster	exactly 200 of 400 concurrent calls; plan changes reaching two instances; leased quota on four instances; a killed subscription coming back; merchants end to end

Kind	Count	Against	Examples
Control plane (Python)	48	the cluster and the real limiter	the API's isolation and idempotency, invoices and the payment sandbox, signed webhooks to pinned public addresses, TLS, the gateway example

Some tests do more than check examples:

- **A seeded harness for the bound** (3.5.4). One to four engines share a fake Redis that answers in a seeded random order, fails 2% of the calls, and runs on a fake clock with jumps. The theorem is checked over every window. It found two violations in the first protocol, and now passes 100,000 seeds.
- **Property tests against a reference model.**
  - **The histories:** random sequences of checks, retries, plan changes, Redis outages, clock jumps and UTC rollovers, run through the real engine on a transcription of the scripts.
  - **What must match:** every answer and every usage event must equal a separate model of Chapter 3's rules.
  - **The results:** 20,000 seeds per configuration produced 22.6 million matching checks. Ten bugs planted in the engine, one at a time, were all caught (`property_tests_2026-10-23.md`).
- **Sanitizers.** Every test runs under AddressSanitizer and ThreadSanitizer. TSan sometimes cannot start under the kernel's address randomisation, so the TSan suite runs with it off (`setarch -R`).
- **Coverage.** `scripts/coverage.sh` measures line and branch coverage of `src/` and `include/`: 97.1% of lines. The uncovered lines are defensive checks the scripts never trigger, and paths reachable only by a race between threads.
- **Evaluation scripts.** Each claim of Chapter 5 has a script in `tests/eval/` or `bench/`:
  - quota accuracy on four instances, 20 runs;
  - the burst bound, with a model;
  - flood and cardinality;
  - faults: latency, a silent master, dropped connections and a killed master;
  - the noisy neighbour;
  - the benchmark matrix.

  Each writes its raw output to `results/`, and the tables are generated from it (`tools/matrix_tables.py`, `tools/open_loop_table.py`; the ablation and A/B scripts print theirs).

Chapter 5. Evaluation

This chapter tests the two halves of the thesis claim separately, as Chapter 1 set out. First correctness: quotas are exact under concurrency, retries, plan changes and failures (5.2 to 5.4). Then cost: what the leased design buys in throughput and latency, and what it costs in accuracy and isolation (5.5 to 5.10). Then merchants: isolation, changes, billing, the spending cap, capacity, scale and the cost of tenancy (5.12). Every number comes from a script in the repository, and every table names its evidence file in `docs/thesis/evidence/`.

5.1 Setup

Table 5.1: The test setup.

Item	Value
Machine	laptop, Intel Core i5-12450H (8 cores, 12 hardware threads), 7 GB of RAM given to WSL2
System	Ubuntu 24.04 in WSL2 (kernel 6.6.114.1-microsoft-standard-WSL2), gcc 13.3, `-O3 -march=native`
Redis	7.4.11 in Docker 28.5.1: a development cluster of 3 masters; for fault tests, 3 masters with one replica each; no persistence
Limiter	the `release` build at the commit named in each evidence file, defaults unless stated
Load	`bench/loadgen` (C++, gRPC callback API): closed loop at a fixed number of calls in flight, or open loop at a fixed rate with latency from each call's scheduled time; Python clients for the accuracy tests

The load generator, the limiter and Redis share one laptop. That bounds every throughput figure from below: on separate machines each would have its own cores. It also makes the noise large, often ±10% between runs, so the method follows from it.

- **Repetition:** every cell runs at least 3 times, usually 5, and 20 for quota accuracy; tables give the median with the range.
- **Comparisons:** builds or settings are compared in alternating runs within one session, never against a number from another day.
- **Isolation:** for the matrix, the servers and load generators are pinned to separate cores (`taskset`).
- **Extrapolation:** cluster-wide figures are labelled as estimates (5.13), never as measurements.

5.2 Quota accuracy (evaluation test 1)

Setup. `tests/eval/quota_accuracy.sh`:

- **Instances:** four limiter instances.
- **Customers:** one Free (200 a day) and one Pay (2,000 a day with a 10% grace, so 2,200). Their burst limits are raised so that the quota binds.

- **Load:** 400 and 4,400 calls of the `customer` rule, each with a request id, 64 in flight, spread over the instances.
- **Duplicates:** a tenth of the requests are sent twice at the same moment to two different instances. That is the hardest case for deduplication, because neither instance can know about the other's copy.
- **Repetition:** 20 runs.

Result. All 20 of 20 runs were exact. In every run three independent counts each equal the allowance:

- the request ids that got an allowed answer;
- the counter in Redis;
- the usage events of the four instances.

The 96,000 requests included 9,608 sent twice at once, and none was counted twice. No request id got an allowed answer from one instance and a denial from the other, and no call failed (`quota_accuracy_2026-10-22.md`).

Property tests. Random histories go through the real engine on a transcription of the scripts, under a fake clock. They mix checks, retries, plan changes, outages, clock jumps and UTC rollovers. Each answer and usage event is compared with a reference model of Chapter 3's rules. 20,000 histories per configuration gave 22.6 million checks, every one equal to the model. A second property bounds leased quota for large plans. Over 80,490 simulated days, Redis never granted beyond the plan, and what N instances admitted was what Redis granted, minus at most N × C units still held in leases. The test catches bugs: ten planted in the engine one at a time were all caught (`property_tests_2026-10-23.md`).

5.3 Plan changes, the reset and overage (evaluation tests 2 and 3)

- **Upgrades apply at once.** An upgrade written by the admin tool reached two instances 22 ms after the tool started (`PlanChangeIT`). The new limit applied on the next call and kept the day's usage.

- **A downgrade below today's usage** denies with `remaining = 0`, never a negative number (`QuotaDowngradeBelowUsageDeniesWithoutNegativeRemaining`).

- **One reset.** The day is derived from the Redis clock. The script tests cross a UTC midnight, and the property histories crossed 34,425 rollovers, including steps landing within ±1.5 s of the reset. The local denial ends exactly at the reset.

- **Request 2,001 gets three answers** under the three overage policies, through gRPC (`Request2001GivesThreeAnswersOnThreePolicies`):

Policy	Answer
hard	denied
grace	allowed, free
cap	allowed, billed as overage

The property histories produced 133,772 overage answers, all as the model predicted.

5.4 The burst accuracy bound and its model (evaluation tests 4 and 12)

Setup. `tests/eval/burst_accuracy.sh`:

- **Instances and key:** four instances serve one hot key of a leased bucket, 1,000 tokens refilled at 100 a second.
- **Chunk caps:** 1%, 5% and 10% of the limit, so the worst case N × C is 40, 200 and 400 tokens.
- **Demand:** two shapes, at 2,000 calls a second in all:
  - a burst after an idle period, the worst case the proof allows;
  - steady overload.
- **Measure:** the over-admission is the largest excess, over any window, of what was admitted beyond what any exact bucket could admit.
- **Model:** a simulation of the protocol, written independently of the engine, replays the same calls.

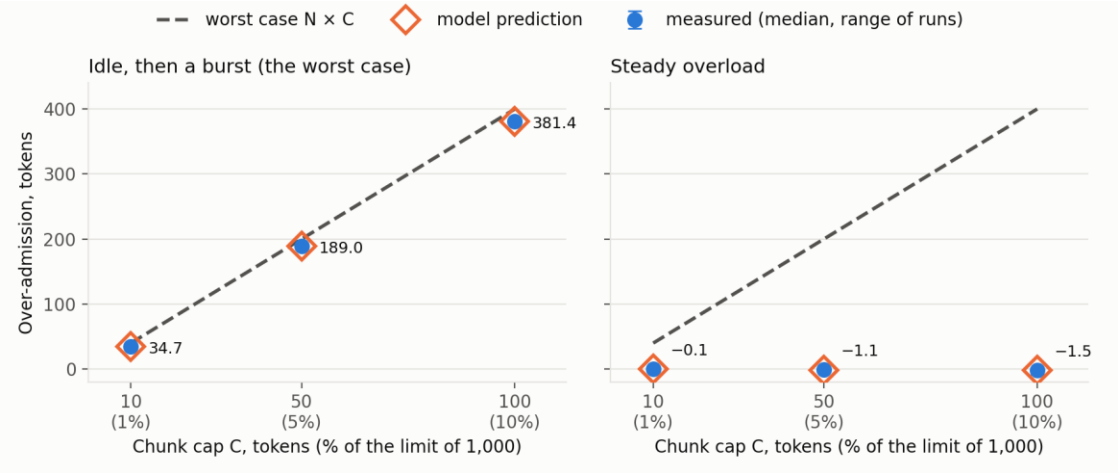


Figure 5.1: Over-admission against the chunk cap: measured (median of 5 runs), predicted, and the worst case N × C (`burst_bound_2026-10-22.md`).

Table 5.2: Over-admission in tokens, median of 5 runs, against the worst case and the model.

Cap	Worst case N × C	After idle: measured	model	Steady overload: measured
1%	40	34.7	34.4	−0.1
5%	200	189.0	188.8	−1.1
10%	400	381.4	381.4	−1.5

- **The bound holds and is tight.** Every run stayed below N × C. After an idle period the excess approaches it: each instance held a full lease, the bucket had refilled, and part of the refill landed within the window.

- **Under steady overload, leases cost no accuracy.** Over a whole run the leases admitted slightly fewer calls than an exact bucket would have.
- **The model predicts the system.** Measured and predicted values agree within 0.3 tokens, and the admitted counts are identical in 29 of 30 runs.

5.5 Floods and many customers (evaluation tests 5 and 6)

A flood (`flood_cardinality_2026-10-22.md`). A Free customer out of quota retried flat out, and the instance denied it 74,881 times a second, its full rate for any check. Those denials cost Redis 2 calls in 10 seconds, both of them plan re-reads, because the deny cache answers locally. At a fixed 10,000 calls a second a denial took 0.24 ms at the median and 0.61 ms at p99.

A million customers.

Table 5.3: One instance over 10,000 and 1,000,000 customers.

	10,000 customers	1,000,000 customers
customer rule	38,201 checks/s	28,365 (−26%), 0.89 plan reads per check
leased rule	72,902 checks/s	47,320 (−35%), 0.62 refills per check, 224 MB of leases

Redis memory per active customer is about 420 bytes: a plan of 171 bytes, plus a daily counter and a burst bucket of 112 each by `MEMORY USAGE`. Uniform keys over a million customers are the hard case for both caches: a customer's next call comes after its plan entry has expired, and a lease is used about once. Skewed traffic, measured in 5.6, keeps both warm.

5.6 Strict against leased, and scaling (evaluation tests 7 and 8)

Setup. `bench/run_matrix.sh` covers:

- **Engines:** no limiter, strict, leased at three chunk caps, and the customer check.
- **Scale:** 1, 2 and 4 instances.
- **Keys:** uniform over a million keys, Zipf over a million keys (exponent 0.99, as YCSB [24]), and one hot key.
- **Load:** closed loop at 192 calls in flight in all, 5 runs per cell; servers and load generators on separate cores.

The tables are generated by `tools/matrix_tables.py` from the 240 runs, none with a failed call (`matrix_2026-10-24.md`).

Table 5.4: Checks per second on one instance, median of 5 runs (p99).

Engine	Uniform, 1M keys	Zipf 0.99, 1M keys	One hot key
gRPC alone	82,704 (5.5 ms)		
strict	42,065 (13.2 ms)	40,571 (12.6 ms)	18,386 (20.3 ms)
leased, cap 1%	42,073 (12.6 ms)	62,679 (7.2 ms)	82,923 (6.0 ms)

Engine	Uniform, 1M keys	Zipf 0.99, 1M keys	One hot key
leased, cap 5%	38,984 (15.7 ms)	66,931 (6.9 ms)	95,605 (4.1 ms)
leased, cap 10%	33,885 (17.9 ms)	67,459 (6.9 ms)	95,913 (3.9 ms)
customer check	25,867 (15.7 ms)	31,519 (13.2 ms)	(three states, below)

Leases pay exactly where keys repeat.

- **Uniform keys over a million:** a run makes about 600,000 checks, so a key is seen less than once, nearly every check misses its lease, and leased equals strict.
- **Zipf:** leases add 55% to 66% and halve p99.
- **One hot key:** they reach 4.5 to 5.2 times strict, as fast as gRPC with no limiter within noise.
- **The chunk cap:** a larger cap is a little faster on repeated keys, because it needs fewer refills. It is the price of the tighter bound of 5.4; on uniform keys the caps differ within noise.

The strict path's hot key is bound by one connection, not by Redis. One instance sends a key's calls through one client, with one connection and one I/O thread: 18,386 checks a second. Two instances reach 46,283 on the same master.

Instances. The instances share the 192 calls in flight and the 6 server CPUs, so on one laptop more instances cannot add capacity:

- **Two instances** beat one on most cells (gRPC alone: 82,704 to 108,946), because one process does not fill its 6 CPUs.
- **Four** lose again (98,419).
- **No engine degrades with N beyond that,** except the customer check, which loses 28% from 1 to 4 instances against strict's 12%. A likely cause, not measured: each instance has its own plan cache, so each sees a customer 1/N as often.

Capacity with more machines is estimated in 5.11, not measured.

Latency at fixed rates (`bench/open_loop.sh`, open loop: latency from each call's scheduled time; `open_loop_2026-10-24.md`):

Table 5.5: p99 latency at fixed arrival rates, open loop, one instance.

Rule	10,000/s	20,000/s	30,000/s	40,000/s
leased `api_default`, p99	0.58 ms	1.45 ms	2.54 ms	3.30 ms
customer check (Pay), p99	2.54 ms	7.84 ms		

On an earlier build (2026-10-21) a leased check queued beyond 20,000 offered (p99 156 ms at 30,000). The knee is now beyond 40,000, consistent with the throughput gained since (5.10). A leased check stays under 1 ms at p99 at 10,000 a second, against the 2 ms target of Chapter 3.

The customer check on one hot key measured three states of one customer, whose quota carried over between runs:

- **Over its burst, within quota:** 14,365 checks a second, every call a burst script on one master.
- **Crossing the quota.**
- **Out of quota:** 84,106 to 90,754, answered by the deny cache.

In the crossing run the client counted about 30,000 allowed answers in 12 s, while Redis counted exactly the plan's 2,200 for the day. By the code, the only uncounted allowed answer is the paid plan's fail-open (`UNVERIFIED`), so Redis most likely stalled during that run. Metrics were off and the stall itself was not observed. Eleven repeats crossing the quota, eight with metrics, were exact, with no deadline misses. The load generator now counts `UNVERIFIED` answers, so the matrix records any such run.

One instance under rising concurrency (`bench/concurrency_sweep.sh`, `scaling_2026-10-24.md`; Figure 5.2). Closed loop from 1 to 384 calls in flight, 3 runs each:

- **Throughput never fell as concurrency rose.** Every median is above the one before.
- **The leased engine follows gRPC alone at every point** (104,581 against 105,626 checks/s at 384).
- **The strict engine levels off near 62,500**, each check waiting for a Redis round trip.

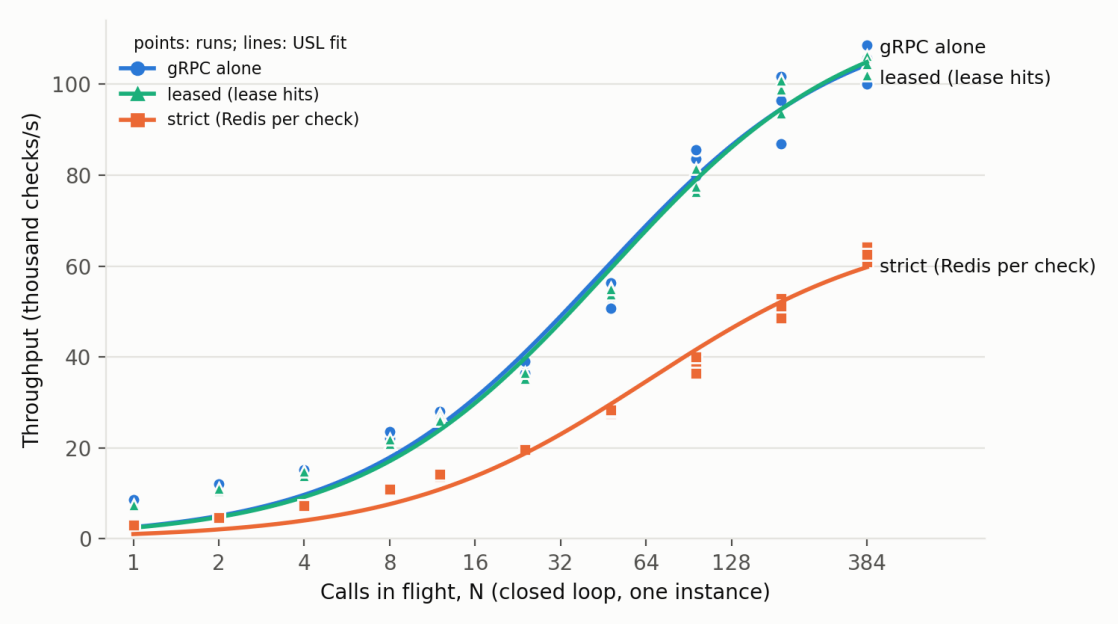


Figure 5.2: Throughput of one instance against calls in flight, with Universal Scalability Law fits.

The Universal Scalability Law [23] does not describe this curve.

- **Least squares** fits the plateau (R^2 0.98, κ about 0, so no retrograde region) but puts the rate at one call in flight at a third of the measured one.
- **A fit on relative error** misses the plateau by up to 49%.
- **The likely reason:** the USL assumes a constant cost per request, while here the cost per call falls as calls pile up, since gRPC and the Redis client send more calls per write and per wake-up.

So the fit gives no projection beyond what the plateau already shows. Little's law [22] holds in all 30 cells: calls in flight ÷ throughput, the mean latency, lies between the measured p50 and p99. So the load generator's latencies account for the whole time a call is out.

5.7 Faults (evaluation test 9)

`tests/eval/faults.sh` and `bench/fault_kill_master.sh` (`faults_2026-10-23.md`). Each fault is measured before, during and after, with Free, Pay and leased streams. Their rates are 1,000, 5,000 and 10,000 checks a second.

Table 5.6: Faults, and what each plan got.

Fault	What each plan got	Recovery
+20 ms on every master (below the 50 ms timeout)	nothing failed; customer checks p50 1.45 → 42.5 ms (two round trips); leased checks 0.61 → 0.64 ms	at once
+80 ms on one master	its third of the customers: Free `UNAVAILABLE`, Pay `UNVERIFIED`, at p99 under 4 ms; leases served until dry	within 1 s
one master silent (replies lost)	the same	before the next phase
every connection killed 5 times in 10 s	2 to 18 calls followed the policy (those in flight); subscription back in 0.5 s	at once
a master killed, replica promoted	a third of the customers got their policy for 4 to 5 s	4 to 5 s

No call failed in any fault run. The test also found a fault in the design. The Redis client's timeout is an idle timeout per connection, so before the fix a master 80 ms slow under load was never timed out: checks succeeded after about 250 ms, with no fallback. Each Redis attempt now has its own deadline.

5.8 A noisy neighbour (evaluation test 11)

Other tenants ran at 5,000 checks a second while one tenant sent 20,000 a second (`noisy_neighbour_2026-10-23.md`).

Table 5.7: One noisy tenant and everyone else.

Noisy tenant	Others' p99	Redis calls a second on its master

Noisy tenant	Others' p99	Redis calls a second on its master
none	4.5 ms	3,319
over its daily quota	14.4 ms	3,319
over its burst, within quota	31.4 ms	25,233

The others' answers stayed correct in every run.

- **Over quota:** the flood cost Redis nothing; the latency rise is shared CPU on the laptop.
- **Over burst:** each call ran the burst script, and that is the isolation gap the design leaves open (3.6.1).

5.9 Ablation (evaluation test 10)

`bench/ablation.sh` switches one mechanism off at a time and compares it with the full system in alternating runs, on the workload where it should matter (5 runs each, one instance, Zipf keys; `ablation_2026-10-24.md`).

Table 5.8: Throughput without each mechanism.

Mechanism	Workload	Full system	Without it	Change
lease	`api_default`, 1M keys	73,172 checks/s, p99 6.05 ms	51,434, 8.55 ms	**−29.7%**
deny cache	Free customers out of quota	98,128, 3.92 ms	34,706, 13.78 ms	**−64.6%**
plan cache	Pay customers	42,232, 10.17 ms	34,148, 11.09 ms	**−19.1%**
early refill	`api_default`, 1M keys	74,571, 5.79 ms	73,927, 5.79 ms	noise
adaptive chunk	same	73,715, 6.05 ms	74,768, 5.79 ms	noise
cache-line alignment	same	73,208, 6.05 ms	74,904, 6.05 ms	noise
batcher (on, against the default off)	strict path	51,457, 7.84 ms	50,984, 8.93 ms	noise

Three mechanisms carry the throughput:

- the lease;
- the deny cache, which answers a customer out of quota locally;
- the plan cache.

The other four have no measurable effect on one instance's throughput. That is not what they are for:

- *Early refill* serves latency when a lease is about to run dry.
- *The adaptive chunk* serves accuracy and limits stranded tokens.
- *Alignment* is 14 times faster in the lease microbenchmark at 16 threads, but in the server each check also pays for gRPC, which hides it.
- *The batcher* gained nothing in an earlier comparison either (4.3), and stays off.

A first deny-cache run measured nothing. A Free customer's burst of 2 a second cannot reach its quota of 200 within a run, so the cache never acted. The run above starts every Free customer out of quota: an ablation is only as good as its workload.

5.10 Profiling and speed-ups

The final build's profile (`bench/profile.sh`, `profile_2026-10-24.md`; Figure 5.3) was taken with `perf` at 999 Hz with frame-pointer stacks, under the closed loop on one instance, and drawn as a flame graph [25].

Table 5.9: Self time by category under load (perf, 10 s).

Self time	Leased rule (75,289 checks/s)	Customer rule (39,933)
gRPC	53.3%	39.6%
allocation	15.9%	20.3%
Redis client	4.3%	8.6%
the limiter's own code	4.1%	6.3%

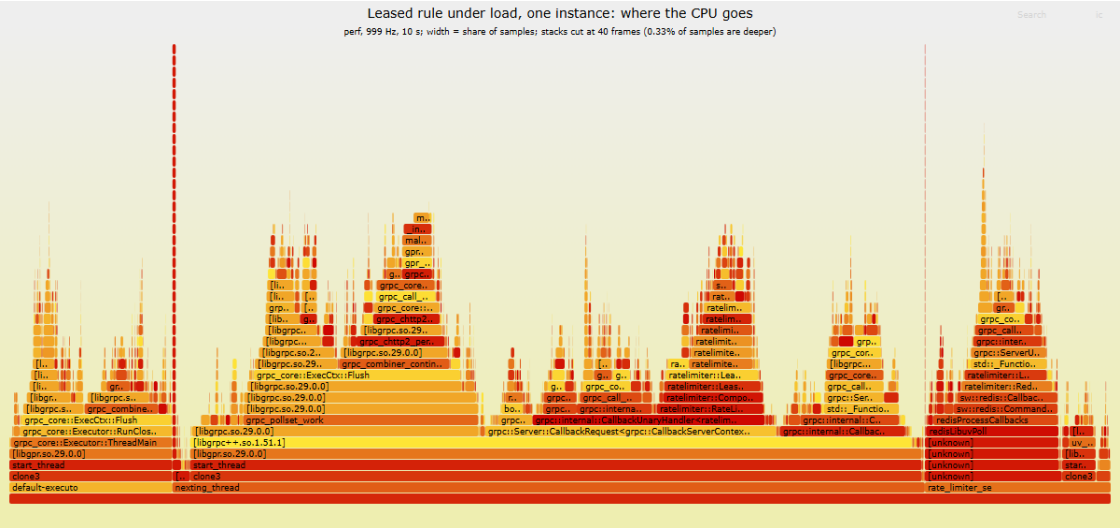


Figure 5.3: Where a leased check's CPU goes. Each bar is a function, its width the share of samples; stacks are cut at 40 frames, which changes no width (0.33% of samples are deeper). The interactive versions are `figures/profile_leased.svg` and `figures/profile_customer.svg`.

The limiter's own code is a small part of the cost. gRPC takes the largest share, plus most of the allocation and all the locks. Allocation is spread thin:

- gRPC's per-call arena is the largest single caller, at 2.8%.
- This project's code allocates about 3% on the customer path and nothing on the lease path beyond gRPC's own.

So the remaining speed-up lies in fewer gRPC calls per check, not in the engine. Measured: `CheckBatch` with 10 checks per call reached 400,187 checks/s on one instance, 5.5 times single calls on the same workload (`matrix_2026-10-24.md`).

The one large finding came from the same method, the day before: abseil's mutex deadlock detection, running in gRPC's own threads. Switching it off gave:

Table 5.10: abseil's mutex deadlock detection on and off, medians of 5 alternating runs.

Workload	Before	After	Change
leased rule	72,097 checks/s	89,743	+24.5%
customer rule	36,469	41,654	+14.2%
gRPC alone	74,733	91,154	+22.0%

Smaller results:

- A lock-free per-attempt deadline queue took a Redis call through the resilient layer from 448 to 214 ns.
- LTO, a larger glibc thread cache and jemalloc changed throughput within noise, so none is adopted.
- PGO was not tried: like LTO it rebuilds only the project's code, 4 to 6% of the CPU.
- Redis Functions (`FCALL`) ran the same script at the same rate as `EVALSHA`, without and with pipelining (62,461 against 62,469 calls/s unpipelined).

5.11 Extrapolation to 1,000,000 checks per second

This section applies the model of Section 3.8 to the final measurements (`tools/extrapolate.py`, which names the source of every input). Its outputs are **estimates, not measurements**: no cluster of that size was run.

Table 5.11: Redis masters at 1,000,000 checks per second, for traffic of one kind of check only; a real mix lies between the rows. A master is taken to absorb 120,000 script calls a second.

All checks are	Redis calls per check	Calls/s	Masters at 120,000
a leased rule, hot keys (10k active)	0.017	17,000	1
a leased rule, 1M keys (Zipf)	0.217	217,000	2
customer checks, enterprise plans (quota leased)	1.017	1,017,000	9
customer checks, Free and Pay, plans cached	2	2,000,000	17
customer checks, Free and Pay, worst case	2.89	2,890,000	25

All checks are	Redis calls per check	Calls/s	Masters at 120,000
for the plan cache			

Table 5.12: Limiter instances at 1,000,000 checks per second.

Operating point	Checks/s per instance	Instances
leased rule, saturated (p99 about 7 ms)	67,459	15
leased rule, p99 below 2 ms (goal G6)	20,000	50
leased rule, CheckBatch of 10	400,187	3
customer check, saturated (p99 about 13 ms)	31,519	32
customer check, at a rate answered in full (p99 7.8 ms)	20,000	50

Reading.

- **Leased rules are cheap in Redis and bound by gRPC.**
  - *Redis:* one or two masters, even when a million keys are active.
  - *Instances:* 15 near saturation, or 50 to keep p99 under 2 ms. That is gRPC's cost per call, since the limiter's own code is 4 to 6% of the CPU (5.10).
  - *Batching:* with batches of 10, three instances would do.
- **The customer check is bound by Redis:**
  - two scripts per check, so 17 masters;
  - 25 if the plan cache is cold for every check. That worst case was measured at a million customers spread uniformly over one instance; with N instances the plan reads grow with N (3.8.2).
- **Leasing the burst limit per customer, the future work of 3.2.1,** would bring customer checks far below their limit towards the leased row.

Cost. Prices are AWS on-demand list prices, retrieved 2026-10-24 from instances.vantage.sh:

- c7i.2xlarge (8 vCPU), $0.357 an hour, for one limiter instance. The laptop gave each instance 6 logical CPUs.
- cache.r7g.large (2 vCPU, 13 GiB), $0.219 an hour, for each Redis master and its one replica.

That an 8-vCPU cloud instance does what the laptop instance did, and a Graviton core what a laptop core did, are further assumptions.

Table 5.13: Monthly cost at 1,000,000 checks per second (estimate).

Deployment	Instances	Masters (+ replicas)	Per month	Per billion checks
leased rule, p99 below 2 ms	50	2 (+2)	$13,670	$5.20
leased rule, CheckBatch of 10	3	2 (+2)	$1,421	$0.54

Deployment	Instances	Masters (+ replicas)	Per month	Per billion checks
customer checks, Free and Pay, answered in full	50	17 (+17)	$18,466	$7.03

Reading the cost table.

- **Most of the cost is limiter instances, that is, gRPC calls.** Batching ten checks per call cuts the leased deployment's cost by about ten times.
- **Memory fits.** If every call carries a request id, about 3 GB of stored answers falls on each of 17 masters (3.8.2), which fits in 13 GiB.

Assumptions (3.8.4), with what the measurements added:

- *A server instance does at least what a laptop instance does.* In the matrix runs the load generator used 1.7 to 5.4 cores of the same laptop (5.6).
- *Instances scale linearly.* Not testable on one laptop. The matrix shows no engine degrading with up to four instances, and the USL fit of a single instance (below) shows where one instance saturates.
- *A master absorbs 120,000 script calls a second.* Redis Functions ran the same script at the same rate as `EVALSHA` (5.10), so the choice of command does not change it.

5.12 Merchants (G10 to G14)

The service was then evaluated with merchants (3.9), on the real limiter (`RL_TENANCY=merchants`, engine `leased`) and the management API, on the development cluster unless stated (`p4_merchants_2026-10-28.md`).

Isolation (G10). Two merchants each had a customer named `cust-1`, on a plan of 5 calls a day. All nine checks passed:

- *Separate customers:* A's customer was refused after its 5 calls, while B's was untouched.
- *Separate management:* each merchant's API showed only its own usage, and A's management key could neither see nor change B's customers.
- *No naming trick:* calls naming `shop-b|cust-1` under A's API key did not reach B's customer.
- *Keys:* an unknown key, and a revoked one, were refused at once.

With a planted bug that put every merchant's customers in one namespace, four of the nine failed. The other five are decided by the API and the key index, which the bug does not touch.

A noisy merchant (G10). Evaluation test 11 (5.8) was repeated with merchants:

- *The others:* another merchant's 1,000 Pay customers at 5,000 checks a second.
- *The noisy merchant:* 20,000 calls a second to one customer.

- *Redis calls:* counted over all three masters. The others' own checks cost about 10,000 a second.

Table 5.14: A noisy merchant and another merchant's customers (medians of 3 runs).

Noisy merchant's flood	Others' p50	Others' p99	Redis calls a second
none	1.12 ms	4.66 ms	10,135
unknown API key	2.14 ms	14.39 ms	10,115
over the operator's check rate	1.65 ms	3.76 ms	10,281
over its own capacity	1.65 ms	3.44 ms	11,575
over its customer's daily quota	1.72 ms	6.32 ms	10,136
over its customer's burst, within quota	2.23 ms	16.38 ms	30,144

The others' calls were all answered and allowed, 50,000 of 50,000 in every run.

- **Four floods cost Redis little or nothing:** an unknown key, a flood over the check rate, over the capacity, or over the quota. The most was 1,440 calls a second above the baseline for 20,000 calls a second of flood, from lease refills of the capacity. The others' latency rose only where the limiter's CPU, shared with the load generators, answered 20,000 more calls a second.
- **A flood over a customer's burst still costs Redis:** one script per call, the gap of 5.8.
  - *The effect:* in one of three runs, 75 of the others' calls were answered by their plan's failure policy instead of a counted decision.
  - *The fix:* the operator's check rate, which holds Redis at its baseline.

Fixed by this test: a flood over the capacity.

- *What the first runs found:* each call over the capacity ran the customer's burst script, then returned its token. That was two Redis calls per refused call, and up to 277 of the other merchant's calls in a run were answered by their failure policy.
- *The change:* while Redis says the capacity is out, the call is refused from the lease alone, before the burst script (4.7).
- *Measured in alternating runs* (Table 5.15).
  - *Its cost on every check of a merchant with a capacity* (one more lease lookup): 32,567 checks a second before and 30,141 after. The ranges overlap, so no difference is shown within the laptop's noise.
  - *Its effect on the result:* nothing is admitted that was refused before, since the early refusal denies only what the lease would. The capacity property test gives the same totals with it.

Table 5.15: A flood over a merchant's capacity, before and after the early refusal (3 alternating runs each).

Build	Redis calls a second	Others' p50	Others' p99

Build	Redis calls a second	Others' p50	Others' p99
before	41,655, 42,539, 42,342	5.09, 5.09, 4.47 ms	44.38, 42.50, 44.38 ms
after	12,489, 12,721, 12,072	2.44, 2.54, 2.05 ms	12.63, 27.55, 14.39 ms

The latency of a change (G11).

- *Setup:* two limiters and a background load of 10,000 checks a second.
- *Each change:* made through the management API. Then both limiters were checked in turn until each answered with it.
- *The customers:* a customer with a leased quota of 1,000,000 a day, and one held by the deny cache.

Table 5.16: From the API's answer to both limiters enforcing a change (30 trials each).

Change	p50	max	API call and propagation, max
suspend, resume	2.19, 3.16 ms	2.86, 4.02 ms	13.3 ms
quota down (a leased quota), up	3.74, 3.21 ms	5.82, 4.92 ms	15.9 ms
boost on (a customer denied), off	4.09, 4.02 ms	6.57, 5.52 ms	14.9 ms
the merchant's capacity on, off	1.20, 2.24 ms	5.77, 4.93 ms	14.6 ms

- **Every change applied within 16 ms,** the API call included, against a goal of 1 s.
- **Plan changes left no stale answers:** once the API answered, neither limiter answered under the old terms, for a lowered leased quota too. The cached plan and denial are dropped on the message, and a lowered quota leaves the leased path (3.4.5).
- **A capacity change gave 1 or 2 old answers** in 10 of 60 trials, since it reaches the limiters by a reload of the merchant table rather than a cache drop.

Billing exactness (G12). The P3 gate was rerun on the final code: one merchant, 40 postpaid customers, 1,500 calls a second for 20 s over two limiters, on the fault cluster (replicas, AOF). The gateway retried with the same request id, and a master was killed 8 s in.

Table 5.17: Invoices against the counters and the gateway's view (`tests/eval/billing_gate.py`).

Run	Calls	Told: in plan / overage	No decision after 6 tries	Counters and invoice: overage	Customers with a difference
no kill	29,997	6,000 / 23,997	0	23,997 ($772.03 in all)	0
master	29,999	6,000 /	815	23,184 ($771.60)	0

Run	Calls	Told: in plan / overage	No decision after 6 tries	Counters and invoice: overage	Customers with a difference
7106 killed		23,184			
master 7105 killed	29,996	6,000 / 23,744	252	23,744 ($771.90)	0

Every invoice equalled the counters, and the counters equalled what the gateway was told, for every customer in every run. Both limiters' usage events agreed too.

- *Lost calls:* a call that got no decision after 6 attempts was neither counted nor billed.
- *Not shown:* a killed master can still lose acknowledged increments that had not reached its replica (3.9.6). These runs did not catch such a write, which is not evidence that none can be lost.

The spending cap (G13).

- *The test:* the property tests ran 2,000 seeds per configuration, 571,482 checks each, every answer equal to a model of the rules. That included about 10,030 answers of billed overage and 2,661 customer months with billed overage per configuration.
- *Leases too:* the leased quota test found Redis granting nothing beyond the cap in 8,146 days.
- *It can fail:* a bug planted in the script's transcription, letting billed overage go one call past the cap, failed all four model tests at seed 2.

Capacity and priority (G14).

- *The merchant's capacity:* 1,000 calls a second, 30% reserved for plans with a fee.
- *The load:* a Free flood at 5,000 a second, and paying customers asking for 100 to 600 a second.

Table 5.18: Calls admitted a second under a Free flood (medians of 3 runs, 10 s each).

Load	Free	Paid admitted / asked	Total
1 limiter, paid 100 a second	686.6	100.0 / 100	786.6
1 limiter, paid 300 a second	656.4	300.0 / 300	956.4
1 limiter, paid 600 a second	624.7	378.5 / 600	1,003.2
2 limiters, paid 300 a second	654.9	300.0 / 300	954.9

- **The capacity held:** at most 1,003.2 calls a second over 10 s. The property test checks every window from 1 ms to 4 s against the capacity plus N × C, over 2,000 seeds; the fullest window reached 91% of its bound.
- **Free calls never took the reserve:** at most 687 a second of their 700.
- **Paying customers kept the reserve, and no more.** Asking for at most the reserve, they lost no call. Asking for 600, they got the 300 of the reserve plus about 78 from the shared part, where they compete with the Free calls.

Many merchants.

- *The setup:* 10,000 merchants of 100 customers each, a million plans, written as the API writes them.
- *Redis:* 193 bytes a customer, the merchant's key, catalogue and customer index included.
- *A limiter:* loaded the 10,000 keys and answered within 0.10 s of starting, and used 118 MiB after the load.
- *Throughput:* 25,749 calls a second with a random merchant and customer on each call. Nearly every call met a customer whose plan was not cached, so it paid a plan read besides its two scripts.
- *A new merchant:* its key worked 36.7 ms after it was announced.

The cost of tenancy. The customer cell of `bench/ab_compare.sh` was run in alternating runs of one build: the customer rule over Zipf 10,000 Pay customers, without tenancy, with it, and with a capacity and a check rate set too high to refuse.

Table 5.19: The cost of tenancy (5 alternating runs each).

Variant	Checks a second, median (range)	Difference	p99
without tenancy	36,009 (31,631 to 38,240)		13.78 ms
with tenancy	36,760 (25,243 to 37,483)	+2.1%	11.59 ms
with tenancy, a capacity and a check rate	34,614 (30,529 to 37,383)	−3.9%	14.39 ms

Tenancy costs less than this setup can measure. The differences are inside the laptop's noise, and the last two runs were slower for every variant.

- *Per call:* the API key costs a hash-map lookup in the thread's cache.
- *Per check of a merchant with limits:* each of the merchant's buckets costs a lease hit.

The portal and the demo.

- *The portal:* a headless browser drove every action of the portal against the API, 22 of 22 checks (4.7).
- *Demo v2:* a Pay customer 30 calls past its quota was admitted as billed overage, and the invoice preview showed the charge. A Free flood at 800 calls a second was held to the merchant's capacity, while a paying customer kept all of its calls.

5.13 Discussion and threats to validity

What the numbers support.

- Quotas are exact: 20 of 20 adversarial runs on four instances, and 22.6 million checks equal to a model.

- The lease's over-admission is bounded by N × C. The bound is reached only after an idle period, and it is predicted by an independent model.
- Where keys repeat, a lease hit costs about 2% over gRPC with no limiter. Where keys almost never repeat, a lease gains nothing over strict (5.6).
- Failures degrade per plan, as documented, without failed calls.
- Merchants are isolated by name, and a flood that a merchant's own limits or the operator's check rate refuse costs the others almost nothing in Redis (5.12).
- A change made through the management API applies on two instances within 16 ms, and invoices equal the counters, also with a master killed (5.12).
- Tenancy costs less than the laptop's noise of ±10% (5.12).

What they do not support yet.

- **Absolute throughput on server hardware.** The load generator shares the laptop.
- **Scaling with instances.** On one laptop the instances share the CPUs, so the matrix shows only that no engine degrades with N (5.6).
- **Behaviour under real network latency.** The 20 ms test is a proxy for it.
- **Billing through any failover.** Three runs with a master killed lost nothing, but asynchronous replication can lose an acknowledged increment, and these runs show only that it was rare there.
- **Many instances for merchants.** The latency of a change and the capacity were measured on two; the bound for N instances rests on the property test.

Threats to validity.

- **One laptop.** Client, limiter and Redis compete for the same cores, so throughput is a lower bound and the noise is large. Comparisons are therefore made in alternating runs.
- **Docker networking on loopback.** Real round trips are longer, and they hurt the exact path more than the leased one (5.7).
- **The fake Redis of the property tests** is a transcription of the scripts. The scripts themselves are tested against a real Redis.
- **Uniform and Zipf keys are models of traffic,** not traces of real traffic.

Chapter 6. Conclusion

This thesis set out to show that a multi-tenant rate limiter can keep quotas exact while answering most checks without a round trip to shared storage. The design splits the two kinds of limit:

- **Daily quotas, which are billed, are decided in Redis by atomic Lua scripts:** exact, deduplicated by request id, and reset once by Redis's clock.
- **Burst limits are served from leases:** tokens an instance takes from Redis in chunks and hands out with one atomic operation. Their over-admission is proved to stay below N × C, the number of instances times the chunk cap.

Between the two sit:

- a deny cache, which answers a customer out of quota locally;
- a plan cache, kept fresh by Redis Pub/Sub;
- a failure policy per plan: paid plans fail open, Free plans fail closed.

It then made the limiter a service for many API owners (merchants), without changing how a check is decided:

- **A merchant's customers are named apart** (`<merchant>|<customer>`), from an API key resolved in memory.
- **Changes are made by a management API** and announced, as plan changes were.
- **Billing totals are kept by the quota script,** in the same atomic step as the decision, and invoices are made from them.
- **A merchant's capacity is one more leased bucket,** with a reserve for paying customers.

6.1 Results against the goals

Table 6.1: The goals of Section 1.3 and what the evaluation found.

Goal	Result	Status
G1 quota never over-admitted	four instances, 20 of 20 adversarial runs exact; 22.6 million checks equal to a reference model; 10 of 10 planted bugs caught (5.2)	met
G2 retries not counted twice	9,608 duplicates sent at once to two instances, each counted once (5.2)	met
G3 plan changes apply at once	an upgrade reached two instances 22 ms after the admin tool started; usage kept (5.3)	met
G4 one reset instant	a full counter from yesterday resets once though two instances meet it at once; 34,425 simulated rollovers (5.3)	met
G5 about 1,000,000	estimated, not measured: 15 to 50 instances for leased rules, depending on the latency target,	estimated

Goal	Result	Status
checks/s on a cluster	and 17 Redis masters for customer checks, about $18,500 a month on demand (5.11)	
G6 p50 < 500 µs, p99 < 2 ms	leased rules: p50 0.43 ms, p99 1.45 ms at 20,000 checks/s per instance. The customer check: p99 1.88 ms at 5,000/s, but p50 0.94 ms, because it needs two Redis round trips (5.6)	met for leased rules; p99 only for customer checks
G7 over-admission at most N × C	proved; 100,000 seeded schedules; measured 34.7, 189.0 and 381.4 tokens against 40, 200 and 400, with a model within 0.3 tokens (5.4)	met
G8 floods refused locally	74,881 denials a second cost Redis 2 calls in 10 s; without the deny cache, throughput falls by 64.6% (5.5, 5.9)	met
G9 documented behaviour when Redis fails	a slow, silent or killed master costs only its own customers their plan's policy; no call failed; recovery in 4 to 5 s after a kill (5.7)	met
G10 merchants isolated	9 of 9 isolation checks, 4 of them failing with a planted shared namespace. Floods of unknown keys, or over a merchant's check rate, capacity or quota, cost Redis at most 1,440 calls a second; the others got every call (5.12)	met; a flood over one customer's burst needs the operator's check rate
G11 a change applies everywhere within 1 s	on both of two instances within 6.6 ms of the API's answer, at most 16 ms with the API call (5.12)	met
G12 invoices equal metered usage	3 runs, 2 with a master killed: invoices equal the counters and the gateway's view for every customer (5.12)	met in every run; a failover can still lose a write (6.2)
G13 overage within the spending cap	2.3 million checks equal to a model, about 10,030 billed overage answers per configuration; a planted cap bug caught at seed 2 (5.12)	met
G14 capacity plus N × C, reserve kept	at most 1,003.2 a second against 1,000; paying customers kept every call up to the reserve; 2,000 seeds within the bound (5.12)	met; beyond the reserve paid calls compete with Free ones

The engineering rules held, measured rather than assumed:

- *The lease hot path* takes no lock and makes no allocation (microbenchmark, profile). The limiter's own code is 4 to 6% of the CPU under load.
- *Every test passes under the sanitizers:* 281 tests in release, AddressSanitizer and ThreadSanitizer, plus 48 tests of the control plane, with 97.1% line coverage.

One intermediate target was not met. A leased instance at four times the strict baseline would be 101,204 checks/s. On the laptop, gRPC with no limiter at all runs at 82,700 to 91,200, so no limiter can reach that on one instance with one check per call. With `CheckBatch` of 10 checks per call, one instance reached 400,187 checks/s (5.10).

What the numbers say about the design:

- **Leases pay where keys repeat:** 1.7 times strict on Zipf keys, 5.2 times on one hot key, and nothing when keys are spread so thin that each is seen once (5.6).
- **The customer check's cost is two Redis round trips.** That cost, not the limiter's code, sets its latency and the number of Redis masters it needs.

6.2 Limitations

The evidence:

- **One laptop.** Load generator, limiter and Redis share 12 hardware threads, and Redis runs in Docker on loopback. Throughput is a lower bound, the noise is often ±10%, and scaling across instances is estimated, not measured. The Universal Scalability Law could not stand in for that measurement: it does not fit one instance's curve (5.6).
- **Synthetic traffic.** Uniform and Zipf keys model traffic; no trace of real traffic was replayed.

The design:

- **One region.**
- **A failover can lose increments** that a master acknowledged but had not replicated, so the promoted replica counts low. Invoices are made from those counters (3.9.6), so such a call would go unbilled, in the customer's favour; the usage events give a second record to reconcile against.
- **Paid plans fail open.** While Redis cannot answer, their calls are allowed uncounted.
- **Burst checks are not deduplicated,** and an over-burst flood within quota costs a Redis call per attempt, which slowed other tenants on the same master (5.8).
- **Plan reads grow with instances times customers,** and leases are never evicted.

Merchants:

- **A flood over one customer's burst still costs Redis,** with merchants as without. It slowed another merchant's checks until the operator set the flooding merchant's check rate (5.12).
- **The reserve guarantees paying customers its own share, not their whole demand.** Beyond it, their calls compete with Free calls for the shared part.
- **The portal shows admitted calls only.** Refusals are not counted per merchant.
- **A merchant's monthly usage is summed per customer** on every request, so its cost grows with the merchant's number of customers.

- **Payments are a sandbox,** in one currency. Billed overage is priced by the plan in force at the end of the month, though a fee that changed is prorated.

6.3 Future work

- **Run the evaluation on separate machines** (planned as a cloud run and not done). This would turn the extrapolation of 5.11 into a measurement and test its assumptions, starting with linear scaling across instances and a master's capacity for the quota script.
- **Lease the burst limit per customer, or deny locally until Redis's retry time.** This closes the noisy-neighbour gap and moves most customer checks to the leased path.
- **Route each customer to one instance** (consistent hashing in the gateway). Plan reads would fall from N × A to A per cache lifetime, and the customer's lease bound from N × C to C.
- **Batch at the gateway.** `CheckBatch` raised one instance's throughput 5.5 times, since gRPC's cost per call dominates.
- **Multi-region limits:**
  - each region with its own cluster and a share of the global quota;
  - shares rebalanced asynchronously, at the price of a bound like N × C between regions.
- **Monthly quota pools,** shared across a customer's projects, beside the daily quota.
- **Lease eviction** for long-running instances over many keys.
- **A default check rate for every merchant,** derived from its plan with the platform, so that a burst flood is refused locally without the operator's action.
- **Running totals per merchant,** sharded across masters, for the portal's numbers and a live count of refusals.
- **A real payment provider** (Stripe's test mode, or VNPay's sandbox with a pay-by-link flow) in place of the sandbox, whose webhook and retries are built.

References

Numbered as cited in the chapters. **V** = content checked online on 2026-10-17/18 while drafting (the quoted statements were read on the page). **L** = well-known literature cited from memory; check the bibliographic details against the original before submission. **B** = bibliographic details checked online on 2026-10-25, cited for its standard result. Web pages: accessed 2026-10-17 unless noted.

1. **V** Amazon Web Services. "Usage plans and API keys for REST APIs in API Gateway." *Amazon API Gateway Developer Guide.* https://docs.aws.amazon.com/apigateway/latest/developerguide/api-gateway-api-usage-plans.html Quoted: "Usage plan throttling and quotas are not hard limits, and are applied on a best-effort basis. In some cases, clients can exceed the quotas that you set."
2. **V** Amazon Web Services. "Throttle requests to your REST APIs for better throughput in API Gateway." https://docs.aws.amazon.com/apigateway/latest/developerguide/api-gateway-request-throttling.html Token bucket throttling; throttles and quotas "should be thought of as targets rather than guaranteed request ceilings."
3. **V** Kong Inc. "Rate Limiting" plugin documentation. https://developer.konghq.com/plugins/rate-limiting/ Policies `local`, `cluster`, `redis`; `redis` is accurate only with `sync_rate` set to 0.
4. **V** Kong Inc. "Rate limiting strategies." https://developer.konghq.com/gateway/rate-limiting/strategies/
5. **V** Envoy Proxy project. *ratelimit* (README). https://github.com/envoyproxy/ratelimit Go/gRPC service implementing Envoy's v3 `rls.proto`; Redis (Memcache experimental) as cache; per second, minute, hour and day limits; `NEAR_LIMIT_RATIO` default 0.8; optional local cache of over-limit keys; shadow mode.
6. **V** Unkey. "Ratelimiting" documentation and engineering notes. https://unkey.com/docs/platform/ratelimiting/introduction , https://engineering.unkey.com/architecture/services/api/ratelimiting . Source: https://github.com/unkeyed/unkey (AGPL-3.0). Sliding window; regional decisions with global convergence within seconds (vendor statement).
7. **V** Upstash. "Ratelimiting algorithms." https://upstash.com/docs/redis/sdks/ratelimit-ts/algorithms Fixed window, sliding window and token bucket over (serverless) Redis; fixed window recommended for multi-region.
8. **V** Cloudflare. "Counting things, a lot of different things." *The Cloudflare Blog.* https://blog.cloudflare.com/counting-things-a-lot-of-different-things/ Sliding window approximation weighting the previous window; "in practice it proved to be good enough."
9. **V** J. Visser, R. Szopa. "Doorman: Global Distributed Client Side Rate Limiting." SREcon16 Europe, USENIX. https://usenix.org/conference/srecon16europe/program/presentation/visser ;

code https://github.com/youtube/doorman (archived 2024). Capacity granted as time-limited leases with periodic refresh; clients cooperate.

10. **V** C. G. Gray, D. R. Cheriton. “Leases: An Efficient Fault-Tolerant Mechanism for Distributed File Cache Consistency.” *Proc. 12th ACM Symposium on Operating Systems Principles (SOSP ’89)*, Litchfield Park, AZ, December 1989, pp. 202-210. ACM, ISBN 0-89791-338-3. Checked 2026-10-18 on a scan of the paper (title page numbered 202, “© 1989 ACM 089791-338-3/89/0012/0202”); pages from dblp. Quoted: “A *lease* is a contract that gives its holder specified rights over property for a limited period of time.” The abstract: leases are “a time-based mechanism”; “Non-Byzantine failures affect performance, not correctness, with their effect minimized by short leases.”
11. **V** Redis. “Scripting with Lua.” https://redis.io/docs/latest/develop/programmability/eval-intro/ Quoted: “Redis guarantees the script’s atomic execution. While executing the script, all server activities are blocked during its entire runtime.” The script cache “is always volatile”; `NOSCRIPT`; keys must be passed as key arguments; effects replication is the only mode from Redis 7.0, which allows `TIME` before writes.
12. **V** Redis. “Redis cluster specification.” https://redis.io/docs/latest/operate/oss_and_stack/reference/cluster-spec/ 16384 slots, `HASH_SLOT = CRC16(key) mod 16384`, hash tags, MOVED and ASK, asynchronous replication (“small windows where acknowledged writes can be lost”).
13. **V** gRPC Authors. “Retry.” https://grpc.io/docs/guides/retry/ Retries only for status codes in the policy’s retryable list, with exponential backoff.
14. **V** gRPC Authors. “Deadlines.” https://grpc.io/docs/guides/deadlines/ and “Status Codes.” https://grpc.io/docs/guides/status-codes/ (accessed 2026-10-18). “By default, gRPC does not set a deadline”; past the deadline the call fails with `DEADLINE_EXCEEDED` (4). Codes `OK` (0), `INVALID_ARGUMENT` (3), `RESOURCE_EXHAUSTED` (8): “Some resource has been exhausted, perhaps a per-user quota, or perhaps the entire file system is out of space”, `UNAVAILABLE` (14).
15. **V** Google. “Language Guide (proto 3).” *Protocol Buffers Documentation.* https://protobuf.dev/programming-guides/proto3/ (accessed 2026-10-18). A field number “cannot be changed once your message type is in use” and “should never be reused”; deleted field numbers must be reserved; an unset numeric field reads as zero; the first enum value “must” be zero and is best named `..._UNSPECIFIED`.
16. **V** M. Thomson, C. Benfield (eds.). “HTTP/2.” RFC 9113, IETF, June 2022 (obsoletes RFC 7540). Abstract: “allowing multiple concurrent exchanges on the same connection”; Section 1: “TCP head-of-line blocking is not addressed by this protocol.”
17. **V** R. Fielding, M. Nottingham, J. Reschke (eds.). “HTTP Semantics.” RFC 9110 (STD 97), IETF, June 2022. Section 10.2.3: servers send `Retry-After` “to indicate how long the user agent ought to wait before making a follow-up request”; the value is an HTTP-date or a number of seconds.

18. **V** M. Nottingham, R. Fielding. “Additional HTTP Status Codes.” RFC 6585, IETF, April 2012. Section 4: “The 429 status code indicates that the user has sent too many requests in a given amount of time (”rate limiting”)“; the response MAY include `Retry-After`.
19. **V** J. Heinanen, R. Guerin. “A Single Rate Three Color Marker.” RFC 2697 (Informational), IETF, September 1999. Three parameters, “Committed Information Rate (CIR), Committed Burst Size (CBS), and Excess Burst Size (EBS)”; the meter uses “two token buckets, C and E, which both share the common rate CIR”
20. **V** Gubernator project (developed at Mailgun). *gubernator* (README and docs/architecture.md). https://github.com/gubernator-io/gubernator (Apache-2.0), read 2026-10-19. Peers hold limits in memory, “doesn’t rely on external caches like memcached or redis”; a consistent hash picks each limit’s owner; `BATCHING` (default 500 µs window) groups peer requests; `GLOBAL` answers on the receiving peer and sends hits to the owner asynchronously: “the use of GLOBAL allows for greater scale but at the cost of consistency”. Production note: “a single gubernator node fields over 2,000 requests a second with most batched responses returned in under 1 millisecond”.
21. **V** Bucket4j project. *bucket4j*, `io.github.bucket4j.distributed.proxy.optimization` (`Optimizations.java`, `DelayParameters.java`). https://github.com/bucket4j/bucket4j (Apache-2.0), read 2026-10-19. `batching()` “combines independent requests to same bucket into batches in order to reduce request count to remote storage”; `delaying()` “can serve requests locally without synchronization with external storage until thresholds are not violated”, the thresholds being `maxUnsynchronizedTokens` and a maximum time between synchronizations; `predicting()` “tries to predict aggregated consumption rate in whole cluster in order to reduce the risk of overconsumption that caused by DelayOptimization”.
22. **B** J. D. C. Little. “A Proof for the Queuing Formula: L = λW.” *Operations Research* 9(3), pp. 383–387, 1961. doi:10.1287/opre.9.3.383. The mean number in a system equals the arrival rate times the mean time spent in it.
23. **B** N. J. Gunther. *Guerrilla Capacity Planning: A Tactical Approach to Planning for Highly Scalable Applications and Services.* Springer, Berlin and Heidelberg, 2007. doi:10.1007/978-3-540-31010-5. The Universal Scalability Law, with one coefficient for contention and one for coherency delay.
24. **V** B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, R. Sears. “Benchmarking Cloud Serving Systems with YCSB.” *Proc. 1st ACM Symposium on Cloud Computing (SoCC ’10)*, pp. 143–154, 2010. Zipfian requests: “some records will be extremely popular (the head of the distribution) while most records will be unpopular (the tail)”. The exponent 0.99 is the YCSB tool’s `ZIPFIAN_CONSTANT` (`core/src/main/java/site/ycsb/generator/ZipfianGenerator.java`, read 2026-10-25); the paper does not state it.
25. **B** B. Gregg. “The Flame Graph.” *Communications of the ACM* 59(6), pp. 48–57, 2016. doi:10.1145/2909476.

26. **V** D. Vyukov. “Intrusive MPSC node-based queue.” 1024cores. http://www.1024cores.net/home/lock-free-algorithms/queues/intrusive-mpsc-node-based-queue (the site redirected away on 2026-10-25). gRPC v1.51.1 credits it in `src/core/lib/gprpp/mpscq.h`: “Multiple-producer single-consumer lock free queue, based upon the implementation from Dmitry Vyukov here”, followed by that URL (read 2026-10-25).